

Fast Monte Carlo Pricing of Asian Options under Rough Fractional Stochastic Volatility

Tiago Flora
Department of Mathematics
University of Chicago
tflora@uchicago.edu

October 8, 2026

Abstract

We study Monte Carlo pricing of a fixed-strike arithmetic Asian call option under the Rough Fractional Stochastic Volatility (RFSV) model [10], in which log-volatility follows a fractional Brownian motion with Hurst exponent $H = 0.10$. The non-Markovian structure of fBM forces every sample path to be drawn from a dense $N \times N$ Gaussian distribution, requiring an $O(N^3)$ Cholesky setup and an $O(MN^2)$ per-path simulation cost. We implement and benchmark an exact dense Cholesky baseline against two algorithms that exploit structure in the covariance matrix: a circulant-embedding FFT method ($O(N \log N)$ per path, exploiting the stationarity of fBM increments), and a global randomized SVD method ($O(Nk)$ per path, exploiting low-rank structure). We show that the circulant embedding is exact for every $H \leq 1/2$, with smallest eigenvalue $\Delta t^{2H}(N^{2H} - (N-1)^{2H})$ in closed form. Over N from 64 to 4000, the FFT and rSVD costs grow linearly in N , while the Cholesky exponent rises from about 1 to 2.3 as the N^2 mat-vec takes over and its factor outgrows the cache. The FFT method overtakes Cholesky at $N \approx 250$ and is $6.7\times$ faster at $N = 4000$; drawing two independent paths from each inverse transform was essential, since random number generation, not the transform, dominates its per-path cost. The rank-32 rSVD method is $9\times$ faster than Cholesky at $N = 4000$ but underprices by 4–8% at every rank tested. That bias follows the fraction of path variance the truncation discards (13% even at $k = 128$), which the usual Frobenius error (1.2%) badly understates. A variance-matched comparison shows that about 70% of the rough-volatility premium on the at-the-money Asian price is due to roughness itself. Two extensions address the remaining weaknesses: restoring the discarded variance as per-step noise removes the low-rank bias, and a control variate built on the geometric Asian payoff, which has a closed-form expectation given each volatility path, cuts the Monte Carlo variance by a factor of 26–29 for every sampler.

1 Introduction

1.1 The option contract

I am interested in pricing fixed-strike, arithmetic-average Asian call options. The payoff at expiry T is

$$C(T) = \max(A(0, T) - K, 0), \quad A(0, T) = \frac{1}{N} \sum_{j=1}^N S_{t_j}, \quad (1)$$

where S_{t_j} is the price of the underlying at time $t_j = jT/N$ and K is the strike. Path-dependence through the average A prevents the Black–Scholes machinery from yielding an exact closed form, making approximations or simulation necessary. Lévy [17] proposed a moment-matching approximation: matching the first two moments of the discrete arithmetic average to a lognormal yields a tractable formula that is accurate under constant-volatility Black–Scholes dynamics, where S_t is lognormal.

The situation is more difficult under Rough Fractional Stochastic Volatility (RFSV) [10], in which $\log \sigma_t = \nu W_t^H$, where W_t^H is a fractional Brownian motion with Hurst exponent $H \approx 0.10$. Under RFSV, S_t is no longer lognormal, and the two-moment Lévy approximation cannot capture the heavy skew and fat tails introduced by a large vol-of-vol ν . Computing even the second joint moment $\mathbb{E}[S_{t_i} S_{t_j}]$ analytically is intractable: it requires the expectation of the integrated volatility path, which depends on the full continuous history of W^H up to each time point.

Monte Carlo sidesteps the distributional intractability by estimating the expectation directly from simulated paths. By the law of large numbers,

$$\frac{1}{M} \sum_{m=1}^M e^{-rT} \max(A^{(m)} - K, 0) \xrightarrow{M \rightarrow \infty} \mathbb{E}^{\mathbb{Q}}[e^{-rT} \max(A - K, 0)], \quad (2)$$

where $\mathbb{Q}$ is the risk-neutral measure and $A^{(m)}$ is the arithmetic average along the m -th simulated path. No closed-form density or moment-matching is required; we need only the ability to simulate correlated log-volatility paths.

1.2 Why rough volatility

Gatheral et al. [10] provide compelling empirical evidence that equity log-realized-variance behaves as fractional Brownian motion with $H \approx 0.10$. Estimating H by ordinary least squares on log-realized-variance increments across five major indices (S&P 500, NASDAQ, DAX, FTSE 100, Nikkei 225) and time scales ranging from one day to one month, they find a consistent Hurst exponent well below $1/2$, with $R^2 > 0.90$ in each case. I take $H = 0.10$ and their vol-of-vol estimate from high-frequency realized variance data [21], $\nu \approx 0.3$. One subtlety concerns units: Gatheral et al. [10] measure time in days, while my engine measures it in years ($T = 1$). Because the log-vol increment variance scales as

$\nu^2 \Delta^{2H}$, the same fit in year units is $\nu = 0.3 \cdot 252^{0.1} \approx 0.52$, which is the value I use. My own variogram fit to non-overlapping 5-day realized variance from `yfinance`, with time in years, gives $H \approx 0.12$ ($R^2 \approx 0.98$ across a handful of lags) and $\nu \approx 0.87$, the latter inflated by measurement noise in weekly squared-return variance.

Standard Brownian motion corresponds to $H = 1/2$, with uncorrelated increments. At $H = 0.10$, volatility paths are far rougher than BM: increments are negatively correlated at all lags, the path is less regular than classical stochastic volatility models, and the realized variance process mean-reverts on short timescales. The short-dated implied volatility smile steepens beyond what classical stochastic volatility models expect, and volatility clustering manifests on timescales an order of magnitude shorter than, e.g., Heston.

The RFSV model I studied is [10]

$$\log \sigma_t = \log \sigma_0 + \nu W_t^H, \quad dS_t = rS_t dt + \sigma_t S_t dW_t^{\mathbb{Q}}, \quad (3)$$

where W_t^H is the fractional Brownian motion of Mandelbrot and Van Ness [18] with covariance $\mathbb{E}[W_t^H W_s^H] = \frac{1}{2}(t^{2H} + s^{2H} - |t-s|^{2H})$. I set $r = 0$ throughout for simplicity (consistent with `params.hpp`); the drift term drops out of all pricing formulas but is retained in the SDE for completeness. I use $H = 0.10$, $\nu = 0.52$, and base volatility $\sigma_0 = 0.2$ throughout, and also explore the sensitivity of option prices to H and ν in Section 6.3. The base level is not a cosmetic choice. An earlier version of this study used $\sigma_0 = 1$ (100% volatility); at $\nu = 0.52$ that payoff is so heavy-tailed that a single path in 10^6 carried 34% of the sample variance, and Monte Carlo error bars were meaningless. At $\sigma_0 = 0.2$ the largest path carries 0.11%, and the payoff standard deviation is stable across batches. Strictly, the payoff variance is infinite for any lognormal volatility model, because $\mathbb{E}[S_t^2]$ contains $\mathbb{E}[\exp(\Delta t e^{2\nu W})]$; in practice the divergence sits so far in the tail that it is never sampled.

1.3 The computational bottleneck

Naïve Monte Carlo for RFSV requires $O(N^3)$ setup work. Because fBM is non-Markovian, there is no forward recursion: the value of W_t^H depends on the entire history of the driving noise, not just its most recent value. Generating each sample path therefore requires drawing from the full N -dimensional Gaussian $\mathcal{N}(0, C)$, where C is a dense $N \times N$ positive-definite matrix with $C_{ij} = \frac{1}{2}(t_i^{2H} + t_j^{2H} - |t_i - t_j|^{2H})$. The standard approach factorizes $C = LL^\top$ via Cholesky decomposition once ($O(N^3/3)$ flops), then generates each path by computing νLz for $z \sim \mathcal{N}(0, I)$ ($O(N^2)$ flops per path, a lower-triangular matrix-vector multiply).

At $N = 252$ trading days and $M = 10,000$ paths the costs are concrete. Cholesky decomposition requires exactly $N^3/3 \approx 5.3$ million floating-point operations (the leading constant $1/3$ compares favourably to $2/3$ for LU); this is a one-time cost. The per-path triangular multiply costs $O(N^2) \approx 63,000$ flops, so the total Monte Carlo loop costs $O(MN^2) \approx 630$ million flops, more than

100 $\times$ the factorization. MC pricing error falls as $O(M^{-1/2})$: with $\sigma_{\text{payoff}} \approx 9.5$ (Section 5.3), $M = 10,000$ paths give a standard error of ≈ 0.095 price units, about 1.8% of the at-the-money price of ≈ 5.3 , and halving it requires four times as many paths.

Trading desks building sensitivity surfaces run parameter sweeps over H , ν , and K , easily multiplying M by factors of 10–100. Intraday risk monitoring at $N \approx 19,656$ (five-minute intervals over 252 trading days of 6.5 hours each: $252 \times 78 = 19,656$) increases the per-path cost by $\sim 6,000\times$ relative to daily steps.

1.4 Approach and scope

I implement and benchmark three algorithms that reduce the cost of fBM path simulation by exploiting structure in the covariance matrix C :

1. **Dense Cholesky** ($O(N^2)/\text{path}$, exact): exploits the positive-definiteness of C to factor it exactly. By “exact simulation” I mean that the method is free of time-discretization bias: it samples the correct fBM distribution for any finite N , unlike Euler–Maruyama approximations. Serves as the correctness baseline against which the other methods are validated.
2. **Circulant embedding & FFT** ($O(N \log N)/\text{path}$): exploits the fact that the increments of fBM (fractional Gaussian noise, fGn) are stationary. Their covariance matrix is Toeplitz, which can be embedded into a circulant matrix, which can be diagonalized by the DFT [7, 22]. For $H \leq 1/2$ the embedding is guaranteed positive [6], so the method is exact on the grid.
3. **Global low-rank rSVD** ($O(Nk)/\text{path}$): exploits the low-rank structure of smooth off-diagonal blocks of C : the off-diagonal blocks admit low-rank approximations. A randomized SVD [14] compresses $C \approx U \text{diag}(S) U^\top$ at rank k , reducing per-path cost from $O(N^2)$ to $O(Nk)$. In practice the implementation is a global approximation of the entire $N \times N$ matrix (including the singular near-diagonal region) rather than of the smooth off-diagonal blocks alone. This is blunter, and must allocate rank budget to the diagonal singularity, which is why the sampler still discards 13% of the path variance at $k = 128$ for $H = 0.10$, and underprices by $\approx 4\%$ (§5.5).

Section 2 develops the mathematical properties of C that each method exploits. Section 3 describes each algorithm and its complexity. Sections 5 and 6 present empirical timing, memory, accuracy, and model validation results. Section 7 adds two extensions: a variance correction for the low-rank sampler and a conditional control variate that applies to all three samplers.

The project does not address quasi-Monte Carlo, model extensions beyond fBM-RFSV (rough Bergomi, multifactor), or GPU parallelism, though Section 8 situates each of these as a natural next step.

2 Mathematical Foundations

2.1 Fractional Brownian motion and its covariance structure

Fractional Brownian motion W_t^H is the unique (up to scaling) continuous Gaussian process with stationary increments and the self-similarity property $W_{ct}^H \stackrel{d}{=} c^H W_t^H$ for all $c > 0$ [18]. Its covariance kernel is

$$\mathbb{E}[W_t^H W_s^H] = \frac{1}{2}(t^{2H} + s^{2H} - |t - s|^{2H}), \quad (4)$$

which is symmetric, positive-definite for $H \in (0, 1)$, and reduces to the standard Brownian covariance $\min(t, s)$ at $H = 1/2$. The self-similarity $W_{ct}^H \stackrel{d}{=} c^H W_t^H$ and the stationarity of increments $W_{t+h}^H - W_t^H \stackrel{d}{=} W_h^H$ are the properties that make fBM a tractable model of rough volatility; §2.2 exploits increment stationarity directly. The critical property for simulation is that fBM is non-Markovian for $H \neq 1/2$: the value of W_t^H depends on the entire history of the driving noise, not just its most recent value. This rules out any forward recursion and forces each sample path to be drawn from a joint N -dimensional Gaussian.

Discretizing $[0, T]$ into N equally spaced points $t_j = jT/N$ yields an $N \times N$ covariance matrix C with $C_{jk} = \mathbb{E}[W_{t_j}^H W_{t_k}^H]$ as in equation (4). The matrix C is dense (every pair of time points is correlated) and positive-definite for all $H \in (0, 1)$ and $N \geq 1$. This density explains the $O(N^2)$ per-path cost: sampling from $\mathcal{N}(0, \nu^2 C)$ by any exact method requires touching all $N^2/2$ entries of C . Algorithm 1 (§3.1) accepts this cost and factorizes C exactly; Algorithms 2 and 3 avoid it by exploiting structure that C possesses.

2.2 Stationarity of fGn and the Toeplitz property

Although the fBM covariance matrix is dense, its increment process has a crucial structural simplification. Define the fractional Gaussian noise (fGn) increments $\delta W_m = W_{(m+1)\Delta t}^H - W_{m\Delta t}^H$ for $m = 0, \dots, N-1$, where $\Delta t = T/N$. Stationarity of fBM increments implies

$$\gamma(k) := \text{Cov}(\delta W_m, \delta W_{m+k}) = \frac{\Delta t^{2H}}{2}(|k+1|^{2H} + |k-1|^{2H} - 2|k|^{2H}), \quad (5)$$

which depends only on the lag $k = |m - n|$, not on the absolute index m [22]. The $N \times N$ covariance matrix of $(\delta W_0, \dots, \delta W_{N-1})$ is therefore Toeplitz: its (m, n) entry equals $\gamma(|m - n|)$, so every diagonal is constant.

Figure 1 illustrates the contrast. Panels (a) and (b) show the $N \times N$ covariance heatmaps of fBM and fGn respectively. The fBM matrix is not Toeplitz: row m of C varies with m because the variance t_m^{2H} grows with time. The fGn matrix has constant diagonals, confirming the Toeplitz property exactly. Panel (c) shows diagonal slices $C[m, m+k]$ vs. lag k for two different rows m : the fGn curves overlap to numerical precision, while the fBM curves diverge.

Algorithm 2 (§3.2) depends on this Toeplitz structure: only Toeplitz matrices embed into a circulant diagonalizable by the discrete Fourier transform. Applying the FFT method directly to the fBM covariance matrix produces negative eigenvalues and pathological path output.

2.3 Circulant embedding and its positivity condition

A Toeplitz matrix T with first row $(\gamma(0), \gamma(1), \dots, \gamma(N-1))$ can be embedded into a $2N \times 2N$ circulant matrix C_{circ} whose first row is the symmetric extension

$$c = (\gamma(0), \gamma(1), \dots, \gamma(N-1), 0, \gamma(N-1), \dots, \gamma(1)). \quad (6)$$

Circulant matrices are diagonalized by the DFT: the eigenvalues of C_{circ} are exactly $\lambda = \text{FFT}(c)$, computable in $O(N \log N)$ operations. To sample, draw $W = a + ib$ with $a, b \sim \mathcal{N}(0, I_{2N})$ independent (so each component has real and imaginary parts that are independent standard normals), scale component-wise by $\sqrt{\lambda_j/(2N)}$, apply the inverse FFT, and take the real part of the first N elements [7]. The imaginary part of the same elements is a second, independent fGn sample: both have covariance $\gamma(|m-n|)$, and their cross-covariance $\sum_j (\lambda_j/2N) \sin(\cdot)$ vanishes because $\lambda_j = \lambda_{2N-j}$. One forward FFT (once per experiment) computes $\{\lambda_j\}$, then each inverse FFT yields two paths: $O(N \log N)$ per path.

The method is valid only if every eigenvalue λ_j is non-negative, so that $\sqrt{\lambda_j}$ is real [22]. For rough H this holds at every N , not just asymptotically. Craigmile [6] proves that the Davies–Harte embedding is non-negative definite for any stationary process whose autocovariance is non-positive at all non-zero lags, a class that includes fGn with $H < 1/2$. The argument is short enough to give in full for the embedding used here. Because $x \mapsto x^{2H}$ is concave for $H < 1/2$, the second difference in equation (5) is negative, so $\gamma(k) < 0$ for every $k \geq 1$. The eigenvalues of the symmetric circulant are

$$\lambda_j = \gamma(0) + 2 \sum_{k=1}^{N-1} \gamma(k) \cos\left(\frac{\pi j k}{N}\right), \quad j = 0, \dots, 2N-1, \quad (7)$$

and because each $\gamma(k)$ is negative, replacing every cosine by 1 can only lower the sum. The smallest eigenvalue is therefore λ_0 , and the sum telescopes:

$$\begin{aligned} \lambda_{\min} = \lambda_0 &= \gamma(0) + 2 \sum_{k=1}^{N-1} \gamma(k) \\ &= \Delta t^{2H} (N^{2H} - (N-1)^{2H}) \approx 2H \Delta t^{2H} N^{2H-1} > 0. \end{aligned} \quad (8)$$

The FFT method is thus exact on the grid for every $H \leq 1/2$ (at $H = 1/2$ the increments are white noise and every $\lambda_j = \gamma(0)$). The C++ pricer checks the sign of every eigenvalue and throws rather than clipping.

Equation (8) also shows that the margin is thin. Relative to the increment variance $\gamma(0) = \Delta t^{2H}$, the smallest eigenvalue is $N^{2H} - (N-1)^{2H}$: 0.24% at $H = 0.10$, $N = 252$ and 0.08% at $N = 1000$, decaying like $N^{-(1-2H)}$. This

Table 1: Complexity of the three fBM path generation methods. N : time steps; M : paths; k : rSVD target rank. “Exact” means the method samples the correct Gaussian distribution on the grid, up to floating-point error; for the FFT method this rests on the positivity result of §2.3.

Method	Setup	Per path	Memory	Accuracy
Dense Cholesky	$O(N^3)$	$O(N^2)$	$O(N^2)$	Exact
Circulant + FFT	$O(N \log N)$	$O(N \log N)$	$O(N)$	Exact
Low-rank rSVD	$O(N^2 k)$	$O(Nk)$	$O(Nk)$	Approximate

mirrors the fGn spectral density, which vanishes at zero frequency when $H < 1/2$. Section 5.4 confirms the formula numerically. The key pitfall is $\gamma(0)$ itself: dropping the $|k - 1|^{2H}$ term at $k = 0$ halves it, which breaks the bound and spuriously makes 28% of the eigenvalues negative at $H = 0.10$. An earlier version of my stability analysis made exactly this mistake and concluded, wrongly, that the FFT method needed eigenvalue clipping.

2.4 Low-rank structure of smooth off-diagonal blocks

The fBM kernel $C(t, s) = \frac{1}{2}(t^{2H} + s^{2H} - |t - s|^{2H})$ has a singularity at $t = s$, since the term $|t - s|^{2H}$ introduces one. For $H = 0.10$ this gives $|t - s|^{0.2}$, whose derivative $0.2|t - s|^{-0.8}$ is unbounded as $t \rightarrow s$. The function is Hölder-continuous with exponent 0.2 but strictly non-differentiable on the diagonal.

Off-diagonal blocks of C (submatrices coupling two disjoint time intervals) lie in the smooth region $t \neq s$. Smooth bivariate kernels generate matrices with rapidly decaying singular values on such blocks [5].

Figure 1(d) confirms this, and locates the difficulty precisely. The off-diagonal block coupling the first half of the grid to the second half is exactly rank 1 at $H = 0.50$, where the kernel is $\min(t, s)$. At $H = 0.10$ its singular values still fall geometrically: below 1% of σ_1 by rank $k = 3$ and below 10^{-4} by $k = 5$. The spectrum of the full matrix C behaves very differently. It decays only algebraically, and more slowly at $H = 0.10$ than at $H = 0.50$. The slow decay that limits a global low-rank approximation therefore comes from the near-diagonal singularity, not from the far field. This is exactly the structure a hierarchical matrix exploits (§8.1).

3 The three algorithms

Table 1 summarizes the three methods. The rest of this section describes each in turn.

3.1 Algorithm 1: Dense Cholesky

The fBM covariance matrix C is symmetric positive-definite for all $H \in (0, 1)$ and $N \geq 1$, which is sufficient for an exact Cholesky factorization $C = LL^\top$.

Algorithm

1. Build $C \in \mathbb{R}^{N \times N}$ analytically from equation (4): $O(N^2)$ flops.
2. Factorize $C = LL^\top$ in place via Cholesky decomposition [12]: $N^3/3$ flops. This is a one-time cost amortized over all M paths.
3. For each path: draw $z \sim \mathcal{N}(0, I_N)$ and compute νLz , a lower-triangular matrix-vector multiply (N^2 flops), which gives the fBM path directly; then simulate the stock price path and accumulate the payoff.

The method is exact. The cost is $O(N^2)$ memory for L (7.6 MB at $N = 1000$, which still fits in the M2's 16 MB L2 cache) and $O(MN^2)$ total work, which dominates at practical M . This method serves as the correctness baseline: prices from the other two methods are validated against it. For the comparison to be fair, the baseline must do only the work it needs. An earlier version of my implementation multiplied by a dense copy of L , zeros included, at $2N^2$ flops per path; §4.6 describes the fix and its effect.

3.2 Algorithm 2: Circulant embedding + FFT (Davies-Harte)

The fBM covariance is not Toeplitz, but the covariance of the fGn increments δW_k is (§2.2). A Toeplitz matrix embeds into a circulant; circulant matrices are diagonalized by the DFT (§2.3).

Algorithm

1. Compute the fGn autocovariance $\gamma(0), \dots, \gamma(N-1)$ and form the symmetric circulant first row $c \in \mathbb{R}^{2N}$.
2. Forward FFT: $\lambda = \text{FFT}(c)$; the eigenvalues are all real (the circulant is symmetric) and positive (§2.3). The code checks the sign and throws if any is negative. $O(N \log N)$ flops, computed once.
3. Pre-compute $s_j = \sqrt{\lambda_j/(2N)}$ for all j .
4. For each pair of paths: draw $W \in \mathbb{C}^{2N}$ whose real and imaginary parts are i.i.d. $\mathcal{N}(0, 1)$, scale $W_j \leftarrow s_j W_j$, and apply the inverse FFT. The real and the imaginary parts of the first N entries are two independent sets of fGn increments; cumsum each to fBM. $O(N \log N)$ flops per path.

FFTW plans for the forward and inverse transforms are created once before the Monte Carlo loop and reused across all M paths [9].

No clipping is needed. By the argument of §2.3, every eigenvalue is positive for $H \leq 1/2$, so the method is exact on the grid at any N [6]. The smallest eigenvalue is 0.24% of $\gamma(0)$ at $H = 0.10$, $N = 252$ (§5.4): small, but positive by construction.

3.3 Algorithm 3: Global low-rank rSVD

Away from the diagonal $t = s$, the fBM kernel is smooth, so off-diagonal blocks of C are well-approximated at low rank (§2.4). A global rank- k approximation $C \approx U \text{diag}(S) U^\top$ replaces C with a compact factor $L_k = U \text{diag}(\sqrt{S}) \in \mathbb{R}^{N \times k}$, dropping the per-path cost from $O(N^2)$ to $O(Nk)$.

Algorithm

1. Build $C \in \mathbb{R}^{N \times N}$ analytically: $O(N^2)$ flops.
2. Compute a rank- $(k + p)$ randomized SVD of C with $q = 2$ rounds of subspace iteration, re-orthonormalizing after every product (Algorithm 4.4 of Halko et al. [14]; see §4.5): $O(N^2k)$ flops. Retain the top- k singular triplets $(U, S, U^\top)$.
3. Form the approximate Cholesky factor $L_k = U \text{diag}(\sqrt{S}) \in \mathbb{R}^{N \times k}$: $L_k L_k^\top \approx C$.
4. For each path: draw $z \sim \mathcal{N}(0, I_k)$, compute $\nu L_k z$ ($O(Nk)$ flops), cumsum to approximate fBM, simulate payoff.

What it sacrifices. Exactness. The sampled paths have covariance $\nu^2 C_k$ rather than $\nu^2 C$, so they are missing the variance $\nu^2 \text{tr}(C - C_k)$ held by the discarded eigenvalues. Because the spectrum of the full matrix decays only algebraically (Figure 1(d)), that loss is large: 21% of the total path variance at $k = 32$ and still 13% at $k = 128$. Less volatility means lower option prices, and Section 5.5 shows the sampler underpricing by 4–8%.

This is a global rSVD, a single rank- k approximation of the entire $N \times N$ matrix. It must allocate rank budget to both the smooth far-off-diagonal blocks and the singular near-diagonal region, which drives up the required k at small H . A true hierarchical H-matrix keeps near-diagonal blocks dense and compresses only far-off-diagonal blocks [13].

3.4 Other algorithms for further exploration

True hierarchical H-matrix A recursive block-tree H-matrix decomposes C into $O(\log N)$ levels of blocks: near-diagonal blocks at each level are kept dense, while well-separated blocks at the same level are approximated at rank proportional to their distance from the diagonal. The resulting structure supports

an H-Cholesky factorization with $O(Nk \log N)$ construction cost and $O(Nk)$ per-path cost, which is better scaling than the global rSVD at large N [13, 2]. The complexity gain is a $\log N$ factor over Algorithm 3; it requires implementing a recursive block-tree and the associated H-matrix arithmetic (§8.1).

Hybrid Scheme for rough Bergomi The Hybrid Scheme of Bennedsen et al. [3] achieves high RMSE reduction over naïve Euler discretization at the same $O(N \log N)$ cost. It does so by exploiting the separability of the Volterra kernel $K(t, s) = (t - s)^{H-1/2}$ near $t = s$ into a singular analytic part and a smooth remainder. However, this kernel governs the rough Bergomi model [1], not the fBM-RFSV model studied here: log-volatility in rough Bergomi is $\int_0^t (t - s)^{H-1/2} dW_s$, a Volterra integral, whereas in RFSV it is simply νW_t^H . Adopting the Hybrid Scheme would require switching model class.

Quasi-Monte Carlo Replacing pseudo-random Gaussian draws with low-discrepancy sequences [20] reduces MC standard error from $O(M^{-1/2})$ to near $O(M^{-1})$, an improvement that can be combined with any of the three algorithms.

4 Implementation Notes

This section documents the non-obvious design decisions in the C++ codebase that are not fully captured by the complexity table in §3.

4.1 fGn autocovariance, not fBM covariance

The circulant embedding requires a stationary covariance sequence. fBM itself is non-stationary: the variance $\mathbb{E}[(W_t^H)^2] = t^{2H}$ grows with t , so row i of the fBM covariance matrix C is different from row $j \neq i$. Attempting to embed C into a circulant and taking the FFT produces a mix of positive and negative eigenvalues even for $H = 1/2$; the method collapses entirely.

The correct input is the fGn autocovariance $\gamma(k)$ of increments (§2.2):

$$\gamma(k) = \frac{\Delta t^{2H}}{2} ((k+1)^{2H} + (k-1)^{2H} - 2k^{2H}), \quad (9)$$

which depends only on lag k . The pricer uses the fGn autocovariance to build the symmetric circulant first row $c = (\gamma(0), \dots, \gamma(N-1), 0, \gamma(N-1), \dots, \gamma(1))$ and then cumsums the resulting fGn increments back into fBM log-volatility. The bug is silent: the program runs and produces plausible-looking paths, just with the wrong covariance structure.

4.2 FFTW plan creation and reuse

FFTW separates plan creation (analyzing the transform size and selecting an algorithm) from execution [9]. The implementation creates two plans once before the Monte Carlo loop:

1. A forward $c2c$ plan of length $M = 2N$ to compute the circulant eigenvalues from the first row c . This plan is executed once and immediately destroyed.
2. A backward $c2c$ plan (`FFTW_BACKWARD`) of length $M = 2N$ to synthesise fGn increments per path. This plan is reused across all M_{paths} calls to `fftw_execute`.

I used full complex-to-complex ($c2c$) transforms rather than the real-input variants (`r2c/c2r`). The Hermitian-symmetry bookkeeping required for the latter, combined with the phase adjustments in the synthesis step, introduces enough notation to obscure the algorithm; $c2c$ keeps the implementation transparent at negligible memory cost ($2N$ vs $N+1$ complex words for the eigenvalue array).

Both plans use `FFTW_ESTIMATE`, which skips empirical tuning. A production deployment could substitute `FFTW_MEASURE` or persist a wisdom file (§4.11) for repeated calls at the same N .

4.3 `std::complex<double>` and the FFTW ABI

A note for the C++ reader: FFTW’s C interface expects `fftw_complex*`, an array of `double[2]`. The per-path sampler keeps its buffers in a `std::vector` of `std::complex<double>` and passes a `reinterpret_cast` of the data pointer. The cast is valid because the C++11 standard guarantees that `std::complex<double>` has the layout of `double[2]`, real part first. The vector lets RAII manage the buffer lifetime without a manual `fftw_free`.

4.4 FFT normalization convention

FFTW’s backward transform is unnormalized: the output of `fftw_execute` on a length- M array equals M times the true inverse DFT. To obtain fGn increments with the correct covariance structure, the per-component scaling factor is

$$s_j = \sqrt{\lambda_j/M}, \quad M = 2N, \quad (10)$$

not $\sqrt{\lambda_j}$. Applying the unnormalized backward transform to the scaled vector $w_j = s_j(a_j + ib_j)$ (with $a_j, b_j \stackrel{iid}{\sim} \mathcal{N}(0,1)$) then yields $\text{Re}(\text{IFFT}(w))_{0:N}$ with covariance $\gamma(|i-j|)$, as required.

4.5 Randomized SVD: subspace iteration

The rSVD implementation (`rsvd.hpp`) draws a Gaussian test matrix $\Omega \in \mathbb{R}^{N \times (k+p)}$ with oversampling $p = 5$ and orthonormalizes the sketch, $Q = \text{orth}(C\Omega)$. It then applies $q = 2$ rounds of subspace iteration,

$$Q \leftarrow \text{orth}(C \text{orth}(C^\top Q)), \quad q = 2 \text{ times}, \quad (11)$$

where `orth(·)` is a thin Householder QR. This is Algorithm 4.4 of Halko et al. [14]. The QR after every product is what separates it from the plain power iteration of their Algorithm 4.3.

Re-orthonormalizing is necessary here, not cosmetic. Without it, the columns of $(CC^\top)^q C \Omega$ are scaled by singular values raised to the power $2q + 1 = 5$. At $N = 500$, $k = 128$ the ratio $(\sigma_1/\sigma_{k+p})^5$ is $\approx 2 \times 10^{15}$, at the limit of double precision, so the directions the sketch is meant to capture are at risk of being lost to roundoff. An earlier version of my code used Algorithm 4.3. Switching changed the Frobenius errors by less than 0.02% (relative) at the ranks tested, but it removes a failure mode at larger k or N . With subspace iteration, the Frobenius error at every rank is within about 2% (relative) of the optimal Eckart–Young truncation computed from the exact eigendecomposition. The slow error decay in §5.5 is therefore a property of C , not of the randomized algorithm.

4.6 In-place Cholesky and the triangular mat-vec

The per-path product νLz needs to touch only the $N(N+1)/2$ non-zero entries of L . My first implementation copied L out of `Eigen::LLT` into a dense `MatrixXd` and computed $L * z$. That is a general mat-vec which also multiplies the zero upper triangle: $2N^2$ flops instead of N^2 , and twice the bytes read per path. It also kept three $N \times N$ matrices alive during the Monte Carlo loop: C , the factorization’s internal copy, and L .

The current implementation factors in place, running `Eigen::LLT` on an `Eigen::Ref` to C so that L overwrites the lower triangle of C , and multiplies through a lower-triangular view of that storage (`triangularView<Lower>`). In isolation at $N = 1000$, the triangular mat-vec takes $\approx 62 \mu\text{s}$ against $\approx 115 \mu\text{s}$ for the dense one. End to end, the Cholesky run at $N = 1000$, $M = 10,000$ drops from 1.61 s to 1.13 s, and resident memory drops from 3×7.6 MB to 7.6 MB. Prices are unchanged to six significant figures, since the random draws and the arithmetic on the non-zero entries are the same. This correction matters for the comparison in §5.1: against the dense baseline the FFT method looked $1.5\times$ faster at $N = 1000$, but against the fair one it was only $\approx 9\%$ faster, until the change in §4.7 restored a $1.7\times$ advantage.

4.7 Two fGn paths per inverse FFT

After the Cholesky fix, the FFT pricer barely beat it at $N = 1000$, and profiling showed why: its per-path cost was dominated not by the transform but by random number generation. In isolation, the $4N$ Gaussian draws for one transform (real and imaginary parts of $2N$ complex variates) take $\approx 60\text{--}70 \mu\text{s}$, against $\approx 8 \mu\text{s}$ for the size- $2N$ inverse FFT. The first version used only the real part of each transform. As §2.3 shows, the imaginary part is a second, independent fGn path, so `FbmSampler::sample_pair()` now returns both. This halves the Gaussian draws per path and cut the $N = 1000$ run from 1.04 s to 0.71 s. The test suite (§4.10) checks that the imaginary-part paths have the fBM covariance and are uncorrelated with the real-part paths.

4.8 Piecewise-constant volatility convention

All three pricers share a single GBM step kernel (`asian_payoff.hpp`):

$$S_i = S_{i-1} \exp\left((r - \tfrac{1}{2}\sigma_i^2) \Delta t + \sigma_i \sqrt{\Delta t} Z_i\right), \quad \sigma_i = \sigma_0 e^{\log \sigma[i]}, \quad (12)$$

where $\log \sigma[i] = \nu W_{t_i}^H$ is the fBM log-vol evaluated at the end of the i -th interval (right-endpoint convention), and $Z_i \stackrel{iid}{\sim} \mathcal{N}(0, 1)$ is an independent Brownian increment for the price process. This departs from the standard Euler-Maruyama left-endpoint rule, which would use σ_{i-1} (the volatility at the start of the interval). The departure does not constitute a look-ahead bias in the causal sense: W^H is pre-generated independently of the price process S , so using σ_i does not introduce any dependence on future values of S . It does introduce an $O(\Delta t)$ discretization error relative to the left-endpoint rule, which at $N = 252$ is negligible. Using the same convention across all three pricers ensures that timing and price comparisons are not confounded by volatility discretization differences.

4.9 Header-only pricers and ODR

Each algorithm lives entirely in one header (`cholesky.hpp`, `fft.hpp` or `lowrank.hpp`), and each `price()` function is marked `inline`:

```
namespace cholesky { inline double price(...) { ... } }
```

`inline` permits the same function to be defined in multiple translation units; the linker deduplicates the definitions and keeps one copy in the final binary. This allows `benchmark.cpp` to include all three headers in one translation unit without ODR violations, while each `_pricer.cpp` standalone executable also includes its header and calls `price()` directly.

4.10 Testing

Several results in earlier drafts of this report came from bugs that nothing checked, so both engines now have test suites that run in a few seconds, gate the benchmark pipeline, and run in continuous integration on every pull request and push to the main branch (Ubuntu with GCC as the gating check, macOS with Apple Clang as an informational one). The C++ suite (`tests/test_samplers.cpp`, 40 checks) verifies that $LL^\top = C$ to 10^{-15} ; that every circulant eigenvalue minimum matches the closed form (8) to 10^{-8} for $H \in \{0.05, 0.10, 0.30, 0.50\}$ and $N \in \{64, 252, 1000\}$; that the sample covariance of 200,000 FFT paths, real and imaginary, matches C within six standard errors per entry, with no correlation between the two halves; that the rSVD error is within 5% of the Eckart–Young optimum, including k close to N ; and that Cholesky and FFT prices agree within Monte Carlo error. Nine further checks cover the extensions of §7, including a negative control that the uncorrected low-rank sampler’s bias is detected, and ten cover the batched pricers of §7.3: the fast generator’s distribution and reference outputs, the covariance of batched paths, a price

that is bitwise identical for any thread count, and a fixed-seed regression value that Apple Clang and GCC reproduce to 10^{-15} . The Python suite (27 tests) checks the same eigenvalue and covariance properties for the numpy engine, the Black-Scholes limit at $\nu = 0$, recovery of known (H, ν) by the variogram calibration, the Oxford-Man data loader on a synthetic file, the control variate, the variance-corrected low-rank sampler, the pathwise vega of §7.4, and that the C++ and Python parameter files agree. A second CI job runs the whole pipeline at small sizes (`./run_pipeline.sh -fast`, about two minutes) and checks that every figure is produced, so analysis scripts cannot break unnoticed.

I checked that the suites catch the bug they were written for: reintroducing the halved $\gamma(0)$ makes seven C++ checks and eight Python tests fail.

4.11 Optimization opportunities

The pricers described so far run one path at a time on one thread, with `std::mt19937`. Three improvements need no change to the algorithms: generating paths in blocks so the core operation becomes a matrix-matrix product, a faster Gaussian generator, and parallelism across paths. I implement and measure all three in §7.3, as opt-in variants, so the benchmarks of §5 are unchanged.

One further option remains untested. The code plans every transform with `FFTW_ESTIMATE`, which picks a plan heuristically. `FFTW_MEASURE` times candidate plans and keeps the fastest, at the price of a slower one-time planning step, and a saved wisdom file pays that price only once across runs [9]. The gain is bounded, though: the inverse FFT is about $8\mu\text{s}$ of a $\approx 70\mu\text{s}$ path at $N = 1000$ (§5.1).

5 Algorithm Benchmarks

All C++ benchmarks were run single-threaded on an Apple M2 MacBook Air (8-core, 24 GB unified memory, rated DRAM bandwidth 100 GB/s; its largest cache is the 16 MB L2 shared by the performance cores, and it has no L3), compiled with `-O3` and `-march=native`, with $M = 10,000$ paths and seven grid sizes N from 64 to 4000 (64, 128, 252, 500, 1000, 2000, 4000). Each timing is the median of five repeats with the same seed, and the benchmark records their interquartile range. The range is under 2% for most configurations (median 0.6%) and reaches 17% for single points. The machine is fanless, and a disturbed run is easy to spot this way: an earlier run whose Cholesky timing at $N = 4000$ was 72% too slow had an interquartile range of 34%, and I discarded it. Benchmarks must also run on mains power: macOS Low Power Mode slowed every timing by $\approx 1.8\times$ in one attempt. Reference prices use $M = 500,000$ paths (see §5.5).

5.1 Runtime scaling

Measurement setup: controlled: $M = 10,000$ paths; independent: N from 64 to 4000, and method; dependent: total wall-clock time.

Table 2: Local scaling exponents $\log(t_2/t_1)/\log(N_2/N_1)$ between successive grid sizes ($M = 10,000$). Theoretical exponents assume the $O(MN^\alpha)$ sampling cost dominates.

N range	64–128	–252	–500	–1000	–2000	–4000	Theory
Cholesky	1.05	1.19	1.28	1.52	1.75	2.26	2.0
FFT	1.09	0.87	1.02	1.02	1.00	1.04	1.0
rSVD ($k = 32$)	0.92	0.84	1.03	1.06	1.08	1.20	1.0

Figure 2 shows the results. A single power law describes the FFT and rSVD methods well, with $\alpha = 1.00$ and 1.02 ($R^2 = 1.000$ and 0.997), but not Cholesky: its global fit gives $\alpha = 1.48$ with $R^2 = 0.983$, because the curve bends. Local exponents between successive grid sizes (Table 2) show what a single exponent hides.

The Cholesky exponent climbs from about 1 to slightly above 2. At small N the per-path cost is not the $O(N^2)$ triangular multiply but the $O(N)$ work around it: drawing N Gaussian variates, computing $2N$ exponentials for the stock price path, and accumulating the payoff. Above $N \approx 1000$ the mat-vec dominates, and between $N = 2000$ and 4000 the exponent exceeds the theoretical 2, reaching 2.26, because L no longer fits in cache (§5.2). The rSVD exponent creeps above 1 at large N as its $O(N^2k)$ construction becomes visible: 24% of the runtime at $N = 4000$.

The FFT method’s $\log N$ factor stays invisible even over this $64\times$ range in N . Figure 3 normalizes the MC time per path by N . An $O(N)$ method is flat in this plot, an $O(N \log N)$ method rises linearly in $\log N$, and an $O(N^2)$ method rises with slope 1. The FFT curve is flat at 70–76 ns per time step from $N = 64$ to 4000. The reason is that the transform is a small part of each path. In isolation at $N = 1000$, the inverse FFT takes $\approx 8 \mu\text{s}$ per transform, while the random numbers and the price-path arithmetic, both $O(N)$, take the rest of the $\approx 70 \mu\text{s}$ per path.

Absolute times matter as much as exponents. The FFT method overtakes Cholesky at $N \approx 250$ and is $1.7\times$ faster at $N = 1000$ (0.71 s against 1.22 s) and $6.7\times$ faster at $N = 4000$ (2.93 s against 19.66 s). The rSVD method is fastest at every N , $9.2\times$ faster than Cholesky at $N = 4000$, but it is approximate (§5.5). Two implementation choices shaped these numbers (§4.6, §4.7): with a dense- L Cholesky baseline the FFT advantage at $N = 1000$ looked like $1.5\times$, and with a triangular baseline but one path per transform it shrank to 9%. A speedup is only as meaningful as its baseline.

5.2 Memory and cache pressure

Controlled: $M = 10,000$ paths; Independent: N , method; Dependent: size of the dominant arrays resident during the MC loop and the measured lifetime peak (C++), peak heap allocation (Python via `tracemalloc`). The measured peak comes from running each method once in a forked child process and reading its

peak resident set size via `wait4`, minus that of an idle child.

Dense methods pay $O(N^2)$ memory. The Cholesky factor occupies $8N^2$ bytes, 7.6 MB at $N = 1000$ and 122 MB at $N = 4000$; the measured lifetime peaks are 11.1 MB and 142 MB, the excess being allocator and matrix-multiply workspace. The FFT method holds only $2N$ scale factors and two length- $2N$ complex buffers, 80 KB at $N = 1000$, and its measured peak at $N = 4000$ is 1.2 MB. The rSVD variant that frees C before the MC loop keeps only L_k resident while sampling (0.24 MB at $N = 1000$, $k = 32$). Its lifetime peak, however, matches the variant that keeps C (both 11.3 MB at $N = 1000$), because C must exist while L_k is built.

The Cholesky loop’s effective streaming rate (Figure 5, right) shows the memory hierarchy at work. Dividing the bytes of the lower triangle of L , read once per path, by wall time gives 14, 24, 33 and 39 GB/s at $N = 252, 500, 1000$ and 2000 , then 33 GB/s at $N = 4000$. The rate rises while the mat-vec takes over from the $O(N)$ work, then falls once every path must stream L from DRAM. That turnover is the signature of a memory-bandwidth-bound loop [8], and it is why the local exponent exceeds 2 at large N . It comes later than the L2 capacity alone predicts ($N \approx 1450$): at $N = 2000$, L is already twice the L2 and the rate is still rising, and this point varies between runs (31 GB/s in an earlier run). At $N = 4000$, L is eight times the L2. The figures remain effective rates, not direct measurements of DRAM traffic; the thread-scaling test of §7.3 gives cleaner evidence that the large- N loop is bandwidth-bound.

The Python RFSV engine (used in Sections 6 and 6.3) is vectorized across paths and so has a different profile. `tracemalloc` on the log-volatility generator measures a peak of 104 bytes per (path, time step), i.e. 13 float64 words per element of the (M, N) output, at every $M \geq 1000$. That count is consistent with the working arrays the generator holds at once, which are $2N$ wide: two real Gaussian arrays and two complex arrays of shape $(M, 2N)$ already account for 12 words per output element. At $M = 10,000$, $N = 252$ the peak is ≈ 260 MB, far beyond any cache. The measurement covers path generation only; the price-path arrays come on top.

5.3 MC convergence

Controlled: $H = 0.10$, $\nu = 0.52$, $\sigma_0 = 0.2$, $K = 100$, $T = 1$, $S_0 = 100$, $r = 0$, $N = 252$; Independent: $M \in \{100, 250, 500, 1000, 2500, 5000, 10000, 25000\}$ and estimator (plain, control variate); Dependent: mean price and cross-seed standard deviation over 20 independent random seeds.

Figure 7 shows the results. The fitted log-log slope of plain Monte Carlo is -0.488 ($R^2 = 0.986$), in good agreement with the theoretical -0.5 from the central limit theorem [11]. The reference on the same grid is 5.3217 ± 0.0018 from 10^6 paths with the control variate (plain: 5.3252 ± 0.0095 from the same paths), and every seed-mean lies within its band. The payoff standard deviation is $\sigma_{\text{payoff}} = 9.46$, so $M = 10,000$ paths give a standard error of ≈ 0.095 , 1.8% of the price. To bring the 95% confidence half-width below 1% of the price would take $M \approx (2\sigma_{\text{payoff}}/0.053)^2 \approx 130,000$ paths.

The control variate keeps the $M^{-1/2}$ rate and lowers the constant. Its per-path standard deviation is 1.82, a variance reduction of 27.1, so the same 1% half-width needs only $\approx 4,700$ paths. Its fitted slope, -0.462 ($R^2 = 0.965$), is noisier than the plain one because each cross-seed standard deviation is itself estimated from only 20 seeds; the per-seed variance reductions range from 14 to 57 around the per-path value of 27. Where differences between configurations matter, I also use common random numbers, which makes those differences far more precise than the absolute prices.

Two earlier versions of this study failed this test, and both failures are instructive. The first estimated σ_{payoff} from the spread of only five seed-level prices, and fitted a slope of -0.71 : a standard deviation estimated from five samples has roughly 35% relative error. The second used $\sigma_0 = 1$, and its payoff was so heavy-tailed that the sample standard deviation grew with the sample size, from 88 over 2×10^5 paths to 118 over 10^6 , with a single path carrying a third of the sample variance. The Monte Carlo error bars of that parameterization were not meaningful; at $\sigma_0 = 0.2$ the standard deviation of 50,000-path batches ranges only from 9.3 to 9.6.

5.4 Numerical stability

(a) FFT eigenvalue positivity: Figure 8(a) sweeps $H \in [0.05, 0.501]$ at $N = 252$ and $N = 1008$ and plots the smallest circulant eigenvalue relative to $\gamma(0)$. No eigenvalue is negative at any H or N tested, so nothing is clipped and the FFT sampler is exact. The minima agree with the closed form (8) to every printed digit: 0.24% at $H = 0.10$, $N = 252$ and 0.08% at $N = 1000$, always attained at λ_0 . The margin collapses as $H \rightarrow 0$, falling to 0.07% at $H = 0.05$, $N = 252$, but it is positive by construction and many orders of magnitude above double-precision roundoff. At $H = 1/2$ the increments are white noise, every eigenvalue equals $\gamma(0)$, and the ratio is 1. Consistent with exactness, the 500,000-path FFT and Cholesky prices in §5.5 agree to within MC noise.

(b) rSVD condition number: Figure 8(b) plots $\kappa(L_k) = \sqrt{S_{\text{max}}}/\sqrt{S_{\text{min}}}$ as a function of rank k at $H = 0.10$, $N = 252$. The condition number grows from 8.2 at $k = 8$ to 23.3 at $k = 64$: as we include smaller singular values (the denominator shrinks), κ grows. A condition number of 23 is well within the range of float64 arithmetic ($\epsilon \approx 10^{-16}$, so $\kappa \cdot \epsilon \approx 2 \times 10^{-15}$) and does not cause numerical issues in path generation.

(c) Cholesky covariance conditioning: Figure 8(c) shows the condition number $\kappa(C) = \lambda_{\text{max}}/\lambda_{\text{min}}$ against N . At $N = 252$, $\kappa(C) \approx 789$; at $N = 1000$, $\kappa(C) \approx 4 \times 10^3$. The fitted power law is $\kappa \approx 1.08 N^{1.19}$: the covariance matrix becomes increasingly ill-conditioned as more time steps are added, but only polynomially. This growth is expected: adding time steps increases correlations between adjacent points, pushing λ_{min} toward zero. The matrix remains positive-definite and Cholesky-factorizable throughout the tested range. **Eigen:**LLT

fails when floating-point roundoff destroys positive-definiteness, which occurs when $\kappa(C) \cdot \epsilon_{\text{mach}} \geq 1$, i.e., $\kappa(C) \geq 10^{16}$. At $N = 1000$ where $\kappa(C) \approx 4 \times 10^3$, this threshold is twelve orders of magnitude away; the fit would not reach it until $N \sim 10^{12}$.

The slow singular value decay visible in Figure 1(d) is a related but distinct phenomenon: a large $\kappa(C)$ means the spectrum spans many decades, but slow decay (many non-negligible eigenvalues) is what actually hinders low-rank approximation. A matrix can have an enormous condition number and still be perfectly approximated at low rank if the eigenvalues drop off rapidly after the first few; in the fBM case, rough H implies both a large condition number and a slow decay rate. As Figure 1(d) shows, that slow decay belongs to the full matrix; its well-separated off-diagonal blocks still compress to rank ≤ 3 .

5.5 Low-rank approximation quality

Controlled: $N = 500$, $M = 100,000$ paths per rank (rSVD) and $M = 500,000$ paths per exact method (reference); Independent: rSVD target rank $k \in \{2, 4, 8, 16, 32, 64, 128\}$; Dependent: Frobenius error, fraction of path variance lost, and relative price error.

Figure 9 shows results. The reference price $p_{\text{ref}} = 5.3144$ is the average of a 500,000-path Cholesky run (5.3195) and a 500,000-path FFT run (5.3093); averaging two independent exact methods avoids privileging either as ground truth. Each run has a standard error of ≈ 0.013 , and their difference of 0.010 is about half the standard error of a difference, consistent with both methods being exact.

With 100,000 paths per rank the price error is resolved, and it is a systematic underpricing: -8.1% at $k = 2$, -6.8% at $k = 8$, -4.3% at $k = 32$, and -3.9% at $k = 128$, each many times the standard error of $\approx 0.56\%$. The Frobenius error suggests a much better approximation than these prices do: it falls from 8.5% to 1.2% over the same ranks.

The key pitfall is that the Frobenius norm is the wrong measure of what matters for pricing. It is dominated by the few large eigenvalues of C , which a rank- k approximation captures. The option price depends instead on how much variance the simulated paths carry, and a rank- k truncation discards the long algebraic tail of small eigenvalues, which hold the short-scale, rough part of the path. The fraction of path variance lost, $\text{tr}(C - C_k)/\text{tr}(C)$, is 37.9% at $k = 2$, 20.8% at $k = 32$, and still 13.2% at $k = 128$; at the first time step a rank-128 approximation keeps only 60% of the variance. Less volatility means a lower Asian price, and the price bias decays slowly in the same way as the variance lost. For low-rank samplers of rough processes, the trace error should be reported alongside, or instead of, the Frobenius error.

6 Model Validation

Three experiments probe the RFSV pricer: (i) the implied volatility smile against live SPY option chain data; (ii) a comparison with constant-volatility baselines, and the roughness premium with and without variance matching; and (iii) a parameter sensitivity sweep over H and ν . Each uses the Python RFSV engine (`data/rfsv_model.py`), which implements the circulant-FFT path generator with the same normalization as the C++ FFT pricer. Every comparison between configurations (different H , ν or K) uses common random numbers, so differences are measured far more precisely than the absolute prices. The Asian prices in experiments (ii) and (iii) use the conditional geometric control variate of §7.2, which cuts the standard error of each price by a factor of about five at $\nu = 0.52$; every number below comes with its standard error.

6.1 Implied volatility smile

Figure 10 shows the results. For this comparison the base level is not fixed at $\sigma_0 = 0.2$ but calibrated per expiration, $\mu_0 = \log \sigma_{\text{eff}}$, so that the RFSV ATM price matches the market ATM price. This aligns the curves at the money (12.1% and 13.1% for the two expirations, with $\sigma_{\text{eff}} = 0.112$ and 0.119), so the comparison concerns smile shape (slope and curvature), not the vol level.

The model smile has clear curvature but no skew. For the nearer expiration ($T = 0.019$) it has its minimum at the money and rises to $\approx 17.5\%$ at $K/S = 0.91$ and $\approx 15\%$ at $K/S = 1.06$. The SPY smile is strongly skewed instead: above 40% at $K/S = 0.91$, falling to a minimum of $\approx 10.5\%$ near $K/S = 1.02$, and recovering to $\approx 16\%$ at 1.06. The later expiration ($T = 0.038$) has the same shape. In the out-of-the-money call wing the two curves roughly meet, so the vol-of-vol $\nu = 0.52$ produces about the right amount of curvature there. Below the money the model misses the skew entirely. This is expected: the engine draws the price shocks independently of the volatility driver, so the spot-vol correlation is $\rho = 0$, and with $\rho = 0$ a stochastic volatility model produces a smile that is symmetric in log-moneyness. The steep short-dated skew that rough volatility is known for [10] requires $\rho < 0$ (§8.5).

A Monte Carlo pitfall: forward error masquerading as skew. An earlier run of this experiment (an October 1 snapshot with $\nu = 0.30$) used $M = 3,000$ paths with common random numbers but no correction, and its RFSV curve rose steadily with strike, from 13.0% to 15.7% across $0.98 \leq K/S \leq 1.05$. That curve looks like an upward skew, which a $\rho = 0$ model cannot produce. The cause was sampling error in the forward. The sample mean of S_T was 0.048% below S_0 , and because every strike shares the same paths, that error acts like a shifted forward and tilts the entire IV curve; it also biases the calibrated μ_0 . A 400,000-path run of the same model gives an approximately symmetric smile with its minimum at the money (14.57% at $K/S = 1.00$, against 14.79% at 0.98 and 14.70% at 1.02). The current script rescales the simulated S_T so that its sample mean equals the forward exactly, a standard moment-matching correction

[11], and uses $M = 20,000$ paths. At $N \approx 5$ time steps per path the whole experiment runs in a few seconds.

Limitations I set $r = 0$ and ignore SPY’s dividend yield. For near-the-money options this shifts IV levels slightly and leaves the shape comparison largely unaffected. For deep in-the-money calls the effect is large: the carry in an ITM call price is misread as time value, which inflates those market IVs (many above 40%), so I do not interpret the low-strike wing. SPY options are American-style; for calls on a low-dividend ETF the early-exercise premium is small, and I treat them as European. The comparison is a single-day snapshot; market IV smiles vary substantially day-to-day. Brent’s method stops at a tolerance of 0.01 in μ_0 , about 1% in σ_{eff} .

6.2 Lévy benchmark and roughness premium

Figure 11 shows the results.

Constant-volatility baselines: When $\nu \rightarrow 0$, $\sigma_t \rightarrow \sigma_0$, so the RFSV dynamics reduce to constant-volatility GBM with $\sigma = 0.2$, which the Lévy approximation is designed for [17]. At this volatility it is accurate: at the money it gives 4.625, against 4.617 ± 0.0005 for exact discrete GBM by Monte Carlo (10^5 paths with the control variate, which for constant volatility removes 99.9% of the variance), an overpricing of only 0.2%. All four RFSV curves lie above both baselines, ordered by H , as stochastic volatility and Jensen’s inequality predict. At the money the RFSV prices are 5.459, 5.318, 4.994, and 4.846 for $H = 0.05, 0.10, 0.30$, and 0.50 (standard error ≈ 0.006), so rough stochastic volatility adds 0.70 to the GBM price at $H = 0.10$. (An earlier version of this experiment used $\sigma = 1$, where the Lévy approximation overprices exact GBM by about 6%, enough to put the Lévy curve above the RFSV curves; it is reliable only at realistic volatility.)

Roughness premium: At fixed $\nu = 0.52$, the premium $\hat{p}(H=0.10) - \hat{p}(H=0.50)$ is positive at every strike, rising from 0.176 at $K = 80$ to 0.471 ± 0.004 at the money and falling to 0.314 at $K = 120$. But this compares more than roughness. At fixed ν , $\text{Var}(\log \sigma_t) = \nu^2 t^{2H}$ is larger for small H when $t < 1$, and the expected integrated variance

$$\mathbb{E} \left[\int_0^T \sigma_t^2 dt \right] = \sigma_0^2 \int_0^T e^{2\nu^2 t^{2H}} dt \quad (13)$$

is 18.6% higher at $H = 0.10$ than at $H = 0.50$ ($1.574 \sigma_0^2$ against $1.327 \sigma_0^2$). To isolate roughness, I solve (13) for the ν at $H = 0.5$ that gives the same integrated variance as $(H, \nu) = (0.10, 0.52)$, which is $\nu = 0.651$, and price again with the same random numbers. The variance-matched premium is 0.332 ± 0.005 at the money, so $70.3 \pm 0.5\%$ of the premium survives: most of the price increase comes

Table 3: Share of the fixed- ν ATM roughness premium (0.471 ± 0.004) that survives when ν at $H = 0.5$ is normalized three ways. The ATM implied volatility of the rough model is 23.54%. Errors are paired standard errors across 10 seeds (delta method for the share).

Normalization	$\nu(H=0.5)$	ATM IV	Premium	Share
Fixed ν	0.520	21.89%	0.471 ± 0.004	100%
$\mathbb{E}[\int_0^T \sigma_t^2 dt]$	0.651	23.05%	0.332 ± 0.005	$70.3 \pm 0.5\%$
$\int_0^T \text{Var}(\log \sigma_t) dt$	0.671	23.26%	0.306 ± 0.005	$65.0 \pm 0.6\%$
ATM implied vol	0.697	23.54%	0.274 ± 0.006	$58.0 \pm 0.8\%$

from path roughness itself, not from higher total variance. All premia peak at the money, where the call payoff is most sensitive to the shape of the volatility path.

How robust is the 70%? Matching (13) is one reasonable normalization, not the only one. I test two others (Table 3). Matching the integrated log-volatility variance $\int_0^T \nu^2 t^{2H} dt = \nu^2 T^{2H+1}/(2H+1)$ instead gives the closed form

$$\nu(H) = \nu_0 \sqrt{\frac{2H+1}{2} T^{1-2H}}, \quad (14)$$

with ν_0 fixed at the Brownian reference ($H = 1/2$); from $(H, \nu) = (0.10, 0.52)$ it gives $\nu = 0.671$ at $H = 0.5$. The third holds the at-the-money European implied volatility fixed, which is closer to how a trader would compare models. I solve for that ν by conditional Monte Carlo: with $\rho = 0$ the European price given a volatility path is the Black–Scholes price at the path’s root-mean-square volatility, so the estimate has no price-shock noise, and with the $H = 0.5$ volatility paths held fixed it is smooth and increasing in ν , which lets Brent’s method converge cleanly. It uses its own seeds, so the matched $\nu = 0.697 \pm 0.002$ is a constant for the pricing runs.

The share falls from 70% to 65% to 58%, in the order in which the normalizations give more vol-of-vol to the Brownian comparison model. The ATM-IV match is the strictest: it removes every difference that shows up in the at-the-money European price, and still 58% of the Asian premium remains. Roughness therefore accounts for most of the premium under every normalization I tried.

Limitations: The standard errors are paired across 10 seeds, so they reflect how consistently the premium appears under common random numbers; the absolute prices carry the larger standard error of ≈ 0.006 .

6.3 Parameter sensitivity:

Figures 12 and 13 show the results:

- **Price decreases with H , and more so at large ν :** at $\nu = 0.52$ the ATM price falls from 5.464 at $H = 0.05$ to 4.862 at $H = 0.50$. The gap between $H = 0.10$ and $H = 0.50$ is only 0.016 at $\nu = 0.10$ but 0.910 at $\nu = 0.70$: without vol-of-vol there is no volatility path whose roughness could matter. As §6.2 shows, most of this effect is roughness rather than added variance.
- **Price increases with ν :** at $H = 0.10$ the ATM price rises from 4.643 at $\nu = 0.10$ to 5.984 at $\nu = 0.70$. Larger vol-of-vol amplifies σ_t variation, and the increase is convex in ν because $\sigma_t = \sigma_0 e^{\nu W_t^H}$ enters exponentially.
- **The H effect peaks at the money:** the spread between $H = 0.05$ and $H = 0.50$ is 0.23 at $K = 80$, 0.60 at $K = 100$, and 0.40 at $K = 120$ (Figure 13), matching the strike profile of the premium in Figure 11(b).

At the model parameters $H = 0.10$, $\nu = 0.52$ the ATM price in the grid is 5.328 ± 0.018 , within half a standard error of the 10^6 -path reference 5.3217 ± 0.0018 (§5.3). With plain Monte Carlo the same cell was 5.223, one standard error (≈ 0.095) low; the control variate shrinks the common offset that all cells share by a factor of five. The variance reduction falls with vol-of-vol, from 418–776 at $\nu = 0.10$ to 16–29 at $\nu = 0.70$, because the control cancels price-shock noise and the volatility path carries more of the variance as ν grows.

Limitations: All grid points share one random seed. This makes the trends across the grid reliable, but the whole surface carries a common MC offset with a standard error of 0.003–0.03 price units. The sensitivity surfaces are computed on a 6×4 grid ($H \times \nu$) at a fixed $N = 252$; finer grids and higher M are feasible but were not run for this report.

7 Extensions: variance correction, a control variate, and faster sampling

Three weaknesses stand out in the results above: the low-rank sampler underprices at every rank (§5.5), plain Monte Carlo needs $\approx 130,000$ paths for a 1% confidence half-width (§5.3), and the per-path loops leave most of the machine idle: one core, one mat-vec at a time, and a random number generator that dominates the FFT path (§5.1). This section adds one remedy for each, and then uses the control variate to estimate Greeks. All are opt-in methods alongside the original pricers, measured by separate programs (`./build/extensions`, `./build/performance`), so none of the results in the preceding sections change.

7.1 Variance-corrected low-rank sampler

The low-rank sampler underprices because it discards the variance held by the small eigenvalues of C . The simplest repair puts that variance back at each time

step as independent noise:

$$W^H \approx L_k z + D^{1/2} \varepsilon, \quad D = \text{diag}(C - L_k L_k^\top), \quad z \sim \mathcal{N}(0, I_k), \quad \varepsilon \sim \mathcal{N}(0, I_N). \quad (15)$$

Every marginal variance $\text{Var}(W^H(t_m))$ is then exact by construction, while the off-diagonal covariances remain those of C_k . The cost is N extra Gaussian draws and N multiply-adds per path.

The correction removes the bias (Figure 14(a)). The plain sampler's price error ranges from -8.1% at $k = 2$ to -3.9% at $k = 128$. The corrected sampler's error is within one standard error ($\approx 0.56\%$) at every rank, from -0.5% to $+0.3\%$, for about 40% more MC time at $k = 32$ (2.0 s to 2.8 s per 100,000 paths).

The key pitfall is that correct marginals do not mean correct paths. The plain low-rank paths are far too smooth: they miss 93–100% of the increment variance (Figure 14(b)), which is where the roughness lives. The correction's noise is white in levels, so it overshoots and makes the increments too rough instead: the increment-variance error is 119% at $k = 2$, falls to 1.8% at $k = 64$, and rises again to 16% at $k = 128$. The lag-1 increment covariance improves from about 100% wrong to 14–18% wrong at $k \geq 64$. The Asian price depends on the volatility levels σ_t , not on their increments, which is why (15) prices it correctly. A payoff that depends on path roughness itself, such as an option on realized volatility, would need the residual's near-diagonal covariance as well, for example through a banded Cholesky factor (§8).

7.2 Conditional geometric control variate

For constant volatility, the geometric-average Asian call has a closed-form price [16] and is the classic control variate for the arithmetic one [11]. Under RFSV no such closed form exists, but one exists *conditionally* on the volatility path. With $\rho = 0$ the price shocks are independent of σ , so given the path, the log of the geometric average $G = (\prod_n S_{t_n})^{1/N}$ is Gaussian with

$$m = \log S_0 + \Delta t \sum_{j=1}^N w_j \left(r - \frac{1}{2} \sigma_j^2\right), \quad v = \Delta t \sum_{j=1}^N w_j^2 \sigma_j^2, \quad (16)$$

where $w_j = (N - j + 1)/N$ is the share of the averaging window that the j -th shock affects. The conditional expectation of $Y = (G - K)^+$ is then a Black–Scholes formula, $\mathbb{E}[Y \mid \sigma] = e^{m+v/2} \Phi(d_1) - K \Phi(d_1 - \sqrt{v})$ with $d_1 = (m - \log K + v)/\sqrt{v}$, computed per path in $O(N)$.

With V the arithmetic payoff and $C = Y - \mathbb{E}[Y \mid \sigma]$, which has mean zero exactly, the estimator

$$\hat{p}_{\text{cv}} = e^{-rT} \frac{1}{M} \sum_{m=1}^M (V^{(m)} - \beta C^{(m)}) \quad (17)$$

is unbiased for any fixed β . I estimate the variance-optimal coefficient $\beta = \text{Cov}(V, C)/\text{Var}(C)$ on a separate pilot run of 10% of the paths, so the main

Table 4: Conditional geometric control variate ($M = 10,000$). Time to reach a 0.1% relative standard error is extrapolated from the measured standard error and per-path cost, including setup and, for the control variate, the pilot run.

Sampler	Variance reduction		Time to 0.1% SE ($N = 1000$)	
	$N = 252$	$N = 1000$	plain	control variate
Cholesky	27.2	29.3	401 s	16.9 s
Circulant + FFT	28.8	28.1	234 s	10.0 s
rSVD ($k = 32$)	25.7	27.7	128 s	5.7 s
rSVD ($k = 32$) + diag	26.6	25.8	179 s	8.1 s

estimate stays exactly unbiased; the variance is then reduced by the factor $1/(1 - \text{corr}(V, C)^2)$. A single driver implements this for all three samplers.

Table 4 and Figure 15 show the results. The variance falls by a factor of 26–29 for every sampler at both grid sizes, and the time to a given accuracy falls by about the same factor: at $N = 1000$, from 401 s to 17 s for Cholesky and from 234 s to 10 s for FFT. For the purposes of this report, that is a larger saving than any change of sampler: switching from Cholesky to FFT at $N = 1000$ gains $1.7\times$, while the control variate gains about $23\text{--}24\times$ within either.

Two details limit the gain. First, the reduction is set by $\text{corr}(V, C) = 0.981$, giving $1/(1 - 0.981^2) \approx 27$, not by the higher raw correlation of the two payoffs (0.998). Because C has mean zero given the volatility path, it cancels the noise from the price shocks but not the noise from the volatility path itself, and that path-driven part is most of the roughly 4% of variance that remains: $\mathbb{E}[Y \mid \sigma]$ alone carries 2.7% of $\text{Var}(V)$. A second control on the volatility path, or quasi-Monte Carlo (§8.6), would be needed to remove it. Second, a smaller standard error does not fix bias. The plain rSVD sampler with the control variate converges $27\times$ faster to a price that is still about 4% too low; combined with the correction of §7.1, it reaches an accurate price in 8.1 s at $N = 1000$.

Limitations: The conditional closed form (16) requires $\rho = 0$. With spot-vol correlation (§8.5), conditioning on the volatility path no longer leaves the price shocks independent, and the control variate would have to be rebuilt, for instance by conditioning on the volatility’s driving noise instead.

7.3 Batched sampling, a faster generator, and threads

The per-path loops of §5 waste the hardware in three ways. The Cholesky mat-vec Lz reads each element of L once and uses it in a single multiply-add, 2 flops per 8 bytes, so at large N it runs at the speed of memory rather than of the arithmetic units (§5.2). The Gaussian generator, `std::mt19937` with `std::normal_distribution`, costs 15 ns per draw and dominates the FFT path. And every path runs on one of eight cores. I address each with an `opt-in price_batched()` in all three samplers, driven by one shared loop.

Design. Paths are generated in blocks of B . Each sampler fills an $N \times B$ block of log-volatility paths at once: Cholesky with one triangular matrix-matrix product LZ , which reads L once per block instead of once per path; the low-rank sampler with one $N \times k$ by $k \times B$ product; the FFT sampler with one batched FFTW call over $B/2$ transforms. The fast generator combines xoshiro256++ [4] with a 256-layer ziggurat [19], in which about 99% of draws cost one table comparison and one multiply; it draws in 3.8 ns, $4.0\times$ faster. Blocks are also the unit of parallel work: threads take blocks from an atomic counter, block b draws from its own stream seeded by (seed, b) , and the block sums are added in block order. The price is therefore bitwise identical for any number of threads, which the test suite checks. Experiment design: controlled, $M = 10,000$ paths, seed, $k = 32$; independent, N , block size B , generator, and thread count; dependent, Monte Carlo time (median of three repeats) and the time of each stage of a path.

Where the time goes. Figure 16 splits each path into its stages. The generator is a fixed $4.0\times$ win wherever random numbers are drawn. Batching helps only where the transform is memory-bound: it cuts the Cholesky transform by $2.6\times$ at $N = 1000$ and $4.3\times$ at $N = 4000$ (from 1621 to 379 μs), and the low-rank product by $2.6\times$. At $N = 4000$ the batched product runs at $N^2/379\mu\text{s} \approx 42$ GFlop/s, roughly three quarters of the nominal double-precision peak of one performance core (four 128-bit FMA units at 3.5 GHz), while the mat-vec streams L at ≈ 40 GB/s. The FFT transform does not change at all (5.4 against 5.3 μs at $N = 1000$): each transform already works in cache, so a batch has nothing to reuse. After both changes, the price path and its $2N$ exponentials, untouched here, are the largest stage of a low-rank path and about a third of an FFT path.

Speedups on one thread. Figure 17 shows the end-to-end effect. The FFT sampler gains 2.0 – $2.2\times$ at every N , entirely from the generator. The low-rank sampler gains 1.7 – $1.9\times$, mostly from the generator. The Cholesky gain grows with N , from $2.2\times$ at $N = 64$ to $2.3\times$ at $N = 1000$ and $4.0\times$ at $N = 4000$ (19.3 s to 4.85 s), because batching removes the memory penalty: the local exponent between $N = 2000$ and 4000 falls from 2.33 for the original loop to 1.74, below the $O(N^2)$ flop count because the $O(N)$ stages still carry weight. The block size has to be large enough to pay off. At $B = 2$ the triangular product is slower than the mat-vec (127 against 87 μs per path at $N = 1000$), and the gain saturates around $B = 64$. Batching narrows the gap between the methods but does not reorder them: at $N = 4000$ the batched FFT sampler is still $3.5\times$ faster than batched Cholesky (140 against 485 μs per path), down from $6.7\times$.

Thread scaling as a test of the bandwidth argument. Figure 18 gives the most direct evidence for the claim of §5.2 that the large- N Cholesky loop is memory-bound. A compute-bound loop should scale with cores, while a bandwidth-bound one should stall once the cores saturate the memory bus. At $N = 4000$ the unbatched Cholesky loop reaches only $1.8\times$ (at three threads) and

falls back to $1.5\times$ at eight, as extra threads compete for the same bandwidth. The batched version, which reads L once per 64 paths, reaches $3.1\times$ on the four performance cores and $4.1\times$ with all eight. The FFT and low-rank samplers scale similarly whether batched or not, to $3.3\text{--}3.8\times$ on four cores at $N = 500$ and to $4.3\text{--}5.7\times$ on eight at either N . Even at $N = 500$, where L fits in cache, the unbatched Cholesky loop scales worst ($2.7\times$ on four cores), since the performance cores share one L2. The efficiency cores add less than the performance cores, so no curve stays linear past four threads.

Limitations: The machine is a fanless laptop, so sustained runs can throttle: back-to-back $N = 4000$ Cholesky runs slowed by $\approx 8\%$ after 40 s, and multi-threaded runs heat the chip faster. The thread-scaling data come from a run with the machine otherwise idle. A repeat with a background process using one core gave the same qualitative picture (unbatched Cholesky at $N = 4000$ peaking at $2.3\times$ and falling to $1.6\times$), but noisier curves. The stage times are measured in isolation with warm caches, so their sum understates a full path by 20–40%. Batching changes which paths a seed produces, so batched prices agree with the original ones only within Monte Carlo error (checked in the test suite), not to the last digit.

7.4 Greeks with common random numbers

With the control variate and common random numbers, sensitivities become cheap to estimate precisely. I compute two for the ATM Asian call at the model parameters: vega with respect to the vol-of-vol ν , and the sensitivity to the Hurst exponent H (`data/greeks.py`). The point of comparing estimators is a consistency check: two unrelated estimators of $\partial V/\partial \nu$ must agree.

The first is the central difference $[\hat{V}(\nu + h) - \hat{V}(\nu - h)]/(2h)$, with both prices from the same random numbers. Each path’s payoff then moves continuously with ν , so the per-path difference is $O(h)$ and the variance stays bounded as $h \rightarrow 0$; with independent random numbers it would grow like h^{-2} . The second is the pathwise estimator [11], which needs no bump. Since $\partial \sigma_n/\partial \nu = \sigma_n W_{t_n}^H$, differentiating the discrete price path gives

$$\frac{\partial V}{\partial \nu} = \mathbf{1}\{A > K\} \frac{1}{N} \sum_{n=1}^N S_n \sum_{m \leq n} \sigma_m W_{t_m}^H (\sqrt{\Delta t} Z_m - \sigma_m \Delta t), \quad (18)$$

where A is the arithmetic average. The payoff is Lipschitz in A , so the derivative and the expectation can be exchanged and (18) is unbiased. The control variate differentiates too: $C = Y - \mathbb{E}[Y \mid \sigma]$ has mean zero for every ν , so $\partial C/\partial \nu$ does as well, and the closed form (16) differentiates through its mean and variance parameters.

The two agree (Figure 19). The pathwise vega with its control is 2.900 ± 0.035 ; the central difference is 2.892 ± 0.035 at $h = 0.005$, 0.16 standard errors away, and drifts up to 2.909 at $h = 0.1$. Since every bump size reuses the same paths, that drift is the $O(h^2)$ curvature bias, measured more precisely than the error bars

suggest. The differentiated control cuts the variance of the pathwise estimator by a factor of 7, against 27 for prices.

The sensitivity to roughness is $\partial V / \partial H = -2.40 \pm 0.05$, flat for h from 0.0025 to 0.02. There is no simple pathwise form, because H enters through the square root of the circulant spectrum, but with common random numbers the bumped paths move smoothly with H and the finite difference is well behaved. At the model point, lowering H by 0.01 raises the price by 0.024, 0.45%; raising ν by 0.01 raises it by 0.029. In these units the price is about as sensitive to roughness as to vol-of-vol, consistent with the sensitivity surface of Figure 12.

8 Future work

The three algorithms implemented here exploit three structural properties of the fBM covariance matrix: positive-definiteness (Cholesky), stationarity of increments (circulant FFT), and smooth off-diagonal low-rank structure (global rSVD).

8.1 True hierarchical H-matrix

The global rSVD (Algorithm 3) approximates the entire $N \times N$ matrix C at rank k . This forces the rank budget to cover both the smooth far-off-diagonal region and the singular near-diagonal region, where $C(t, s) \sim |t - s|^{2H}$ with $H = 0.10$ is Hölder-0.2 continuous and barely above C^0 regularity. The global approximation cannot specialize, which is why the spectrum of the full matrix decays slowly at small H even though its off-diagonal block compresses to rank ≤ 3 (Figure 1(d)).

A true hierarchical H-matrix separates these concerns by a recursive block-tree decomposition of C [13, 2]. At each level of the tree:

- Near-diagonal blocks (time intervals that overlap or touch) are stored dense. The near-diagonal singularity is not low-rank, so no rank budget is wasted trying to compress it.
- Far-off-diagonal blocks (well-separated time intervals) lie entirely in the smooth region $t \neq s$ and are approximated at low rank proportional to the distance from the diagonal.

The result is an H-Cholesky factorization with $O(Nk \log N)$ construction cost and $O(Nk \log N)$ per-path cost (H-matrix triangular solve traverses the block tree), versus the global rSVD's $O(Nk)$ per-path cost. The H-matrix advantage lies in construction: $O(Nk \log N)$ vs $O(N^2 k)$ for the global rSVD sketch, a factor of $N / \log N$ for $k \ll N$.

For the rough case $H = 0.10$, the off-diagonal block needs rank 3 rather than 2 for 1% accuracy (Figure 1(d)): slightly more than at $H = 0.50$, but far below the ranks the global approximation uses. The H-matrix benefit over the global rSVD grows as N increases beyond the current test range. At $N = 252$,

$O(N^2k) = O(252^2 \cdot 64) \approx 4 \times 10^6$ flops for construction; the H-matrix would reduce this by roughly $\log_2 252 \approx 8$ for equal per-block rank, which becomes worthwhile at $N \geq 1000$.

8.2 Beyond the two extensions

Each extension of §7 leaves one gap. The variance correction gets the marginal variances right but the increments wrong; replacing the diagonal noise by a banded factorization of the residual $C - L_k L_k^\top$ (bandwidth b) would also restore the near-diagonal covariances that carry the roughness, at $O(Nb)$ extra cost per path. The control variate removes the price-shock noise but leaves the volatility-path noise, about 4% of the original variance; a second control built on the volatility path, for instance the integrated variance $\int_0^T \sigma_t^2 dt$ whose mean (13) is known in closed form, could remove part of what remains.

8.3 Beyond one machine: GPUs and remaining per-path costs

The batched pricers of §7.3 are already in the form a GPU needs: the Cholesky and low-rank samplers reduce to one dense product per block (cuBLAS `trmm/gemm`), the circulant sampler to one batched FFT (cuFFT), and the counter-style block streams map onto per-thread generators such as Philox. Two caveats from the CPU measurements carry over. Once the transform is batched and the generator is fast, the price path and its $2N$ exponentials are the largest cost of a low-rank path and about a third of an FFT path, so the payoff loop would have to be vectorized as well. And the fair baseline is the batched, multithreaded CPU code, not the per-path loop, which is already $1.6\text{--}3.7\times$ slower on one thread.

8.4 Hybrid Scheme for rough Bergomi

The Hybrid Scheme of Bennedsen et al. [3] achieves more than 80% RMSE reduction over naïve Euler discretization at $O(N \log N)$ cost by exploiting the separability of the Volterra kernel $K(t, s) = (t - s)^{H-1/2}$ near $t = s$: the singular part is handled analytically and the smooth remainder is convolved via FFT. This is a structurally superior simulation method for the rough Bergomi model of Bayer et al. [1], in which log-volatility is a Volterra integral $\int_0^t (t - s)^{H-1/2} dW_s$.

The RFSV model studied here uses a different representation: $\log \sigma_t = \nu W_t^H$, where W_t^H is a standard fBM (not a Volterra integral with the Bergomi kernel). The two models are different processes with different correlation structures, though both have Hurst exponent H controlling roughness. Adopting the Hybrid Scheme would require switching to the rough Bergomi model class, which has distinct option pricing properties and calibration procedures. The structural insight of separating the kernel into a singular analytic part and a smooth FFT-able remainder is directly relevant to the fBM kernel $C(t, s)$ and motivates the hierarchical treatment in §8.1.

8.5 Spot-volatility correlation

The engine draws the price shocks Z independently of the fBM driving volatility, so the spot-vol correlation is $\rho = 0$. As §6.1 shows, this leaves the model unable to produce the downward skew of equity smiles. Introducing $\rho \neq 0$ requires simulating W^H jointly with the Brownian motion that drives the price. The dense Cholesky sampler can absorb this directly by factoring the $2N \times 2N$ joint covariance, which is explicit when W^H is written as a Volterra integral of that Brownian motion; the circulant embedding would need to be extended to the joint process. This is the single change most needed before the IV comparison can test the model rather than its simplifications.

8.6 Quasi-Monte Carlo path generation

All three algorithms use pseudo-random Gaussian draws, giving a standard MC error of $O(M^{-1/2})$. Replacing these with low-discrepancy sequences [20] reduces the effective error rate to approximately $O(M^{-1}(\log M)^N)$, close to $O(M^{-1})$ in practice [11]. This improvement is orthogonal to the choice of covariance simulation method: any of the three algorithms can be combined with QMC by replacing the standard normal draws.

In $N = 252$ dimensions the low-discrepancy property is strongest for the first few coordinates. Brownian bridge dimension ordering concentrates the largest variance components in the lowest-index dimensions, ensuring the low-discrepancy sequence acts on the high-information components first [15]. At $M = 10,000$, the MC standard error of ≈ 0.095 (§5.3) would fall to ≈ 0.001 under ideal $O(M^{-1})$ QMC, a $100\times$ reduction at no additional computational cost, improving the statistical resolution of the roughness premium and parameter sensitivity results.

References

- [1] Christian Bayer, Peter Friz, and Jim Gatheral. Pricing under rough volatility. *Quantitative Finance*, 16(6):887–904, 2016. doi: 10.1080/14697688.2015.1099717. Introduces the rough Bergomi model (Volterra fBM kernel); the Hybrid Scheme of Bennedsen et al. applies to this model class, not to the fBM-RFSV model used here.
- [2] Mario Bebendorf. *Hierarchical Matrices: A Means to Efficiently Solve Elliptic Boundary Value Problems*, volume 63 of *Lecture Notes in Computational Science and Engineering*. Springer, 2008. doi: 10.1007/978-3-540-77147-0. H-LU decomposition and hierarchical preconditioners; complementary reference for the block-tree H-matrix future-work discussion.
- [3] Mikkel Bennedsen, Asger Lunde, and Mikko S. Pakkanen. Hybrid scheme for Brownian semistationary processes. *Finance and Stochastics*, 21(4): 931–965, 2017. doi: 10.1007/s00780-017-0335-5. Hybrid simulation scheme

with better accuracy for rough kernels ($H < 0.5$) at the same $O(N \log N)$ cost; identified as future work to replace the circulant-FFT method at small N .

- [4] David Blackman and Sebastiano Vigna. Scrambled linear pseudorandom number generators. *ACM Transactions on Mathematical Software*, 47(4):36:1–36:32, 2021. doi: 10.1145/3460772.
- [5] Emmanuel Candès, Laurent Demanet, and Lexing Ying. A fast butterfly algorithm for the computation of Fourier integral operators. *Multiscale Modeling & Simulation*, 7(4):1727–1750, 2009. doi: 10.1137/080734339. Theoretical basis for low-rank off-diagonal blocks in smooth kernel matrices; motivates the global low-rank rSVD approach.
- [6] Peter F. Craigmile. Simulating a class of stationary Gaussian processes using the Davies–Harte algorithm, with application to long memory processes. *Journal of Time Series Analysis*, 24(5):505–511, 2003. doi: 10.1111/1467-9892.00318. Proves the Davies–Harte embedding is non-negative definite for stationary processes with non-positive autocovariance at all non-zero lags, which includes fGn with $H < 1/2$; the guarantee that the FFT pricer is exact at $H = 0.10$.
- [7] R. B. Davies and D. S. Harte. Tests for Hurst effect. *Biometrika*, 74(1):95–101, 1987. doi: 10.1093/biomet/74.1.95. Original circulant embedding idea for exact simulation of stationary Gaussian processes; theoretical foundation of the FFT (Davies–Harte) path generator in `src/fft/fft.hpp`.
- [8] Ulrich Drepper. What every programmer should know about memory. Technical report, Red Hat, Inc., 2007. URL <https://www.akkadia.org/drepper/cpumemory.pdf>. Cache hierarchy, memory bandwidth, and latency; used to contextualize the Cholesky bandwidth-bound profile and the L3 spill analysis in the memory benchmark.
- [9] Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005. doi: 10.1109/JPROC.2004.840301. FFTW plan reuse design; justifies creating plans once and calling `fftw_execute` in the MC loop.
- [10] Jim Gatheral, Thibault Jaisson, and Mathieu Rosenbaum. Volatility is rough. *Quantitative Finance*, 18(6):933–949, 2018. doi: 10.1080/14697688.2017.1393551. First circulated 2014. Empirical evidence that log-volatility behaves as fBM with $H \approx 0.10$; defines the RFSV model.
- [11] Paul Glasserman. *Monte Carlo Methods in Financial Engineering*, volume 53 of *Stochastic Modelling and Applied Probability*. Springer, 2003. doi: 10.1007/978-0-387-21617-1. Standard reference for MC pricing, multivariate Gaussian simulation (Ch. 2), and quasi-Monte Carlo (Ch. 5).

- [12] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4 edition, 2013. Cholesky factorization complexity ($N^3/3$ flops, Ch. 4); reference for leading constants and numerical stability.
- [13] Wolfgang Hackbusch. *Hierarchical Matrices: Algorithms and Analysis*. Springer Series in Computational Mathematics. Springer, 2015. doi: 10.1007/978-3-662-47324-5. Full recursive H-matrix theory with block-tree construction and H-Cholesky; identified as future work to replace the global rSVD approach in `src/rsvd/lowrank.hpp`.
- [14] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011. doi: 10.1137/090771806. Algorithm 4.4 (randomized SVD with power iteration) is implemented in `src/rsvd/rsvd.hpp`.
- [15] Corwin Joy, Phelim P. Boyle, and Ken Seng Tan. Quasi-Monte Carlo methods in numerical finance. *Management Science*, 42(6):926–938, 1996. doi: 10.1287/mnsc.42.6.926. Brownian bridge dimension ordering for QMC in high dimensions; identified as future variance-reduction work.
- [16] A. G. Z. Kemna and A. C. F. Vorst. A pricing method for options based on average asset values. *Journal of Banking & Finance*, 14(1):113–129, 1990. doi: 10.1016/0378-4266(90)90039-5. Closed-form geometric Asian option price under geometric Brownian motion and its use as a control variate for the arithmetic Asian; the basis of the conditional control variate in Section 7.
- [17] Edmond Lévy. Pricing European average rate currency options. *Journal of International Money and Finance*, 11(5):474–491, 1992. doi: 10.1016/0261-5606(92)90013-n. Analytical approximation for arithmetic Asian calls under constant-sigma GBM; used as the benchmark in `data/validate_asian.py` (Phase 2 validation).
- [18] Benoit B. Mandelbrot and John W. Van Ness. Fractional Brownian motions, fractional noises and applications. *SIAM Review*, 10(4):422–437, 1968. doi: 10.1137/1010093. Original definition of fractional Brownian motion and fractional Gaussian noise; foundational reference for the covariance kernel implemented in `src/common/covariance.hpp`.
- [19] George Marsaglia and Wai Wan Tsang. The ziggurat method for generating random variables. *Journal of Statistical Software*, 5(8):1–7, 2000. doi: 10.18637/jss.v005.i08.
- [20] Harald Niederreiter. *Random Number Generation and Quasi-Monte Carlo Methods*, volume 63 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. SIAM, 1992. doi: 10.1137/1.9781611970081. Foundational reference for low-discrepancy sequences; identified as future work for variance reduction.

- [21] Oxford-Man Institute. Oxford-Man institute realized library, 2023. High-frequency realized variance estimates for major equity indices. Used for calibrating H and ν via `data/calibrate.py`. Free alternative: `-source yfinance` gives $H \approx 0.117$ vs Oxford-Man $H \approx 0.10$.
- [22] Andrew T. A. Wood and Grace Chan. Simulation of stationary Gaussian processes in $[0, 1]^d$. *Journal of Computational and Graphical Statistics*, 3(4):409–432, 1994. doi: 10.1080/10618600.1994.10474655. General circulant embedding framework for exact simulation of stationary Gaussian processes, including the non-negativity condition on the embedding eigenvalues.

Reproducibility

All source code, benchmark scripts, and plotting code are available at <https://github.com/t-flora/fbm-fast-pricers>. Results can be reproduced end-to-end by running `./run_pipeline.sh` from the project root (`-fast` gives a smoke test in a few minutes). Each run writes a manifest recording the git commit, parameters, and wall time to `plots/runs/TIMESTAMP/manifest.txt`.

Acknowledgements

Portions of this report were drafted and edited with the assistance of Claude Sonnet 4.6 (Anthropic, 2026), Claude Opus 5.5 (Anthropic, 2026), and Gemini 3.1 Pro (Google, 2026).

Structural analysis: why FFT and low-rank rSVD work
 $H = 0.1$, $N = 64$ (heatmaps), $N = 128$ (SVD)

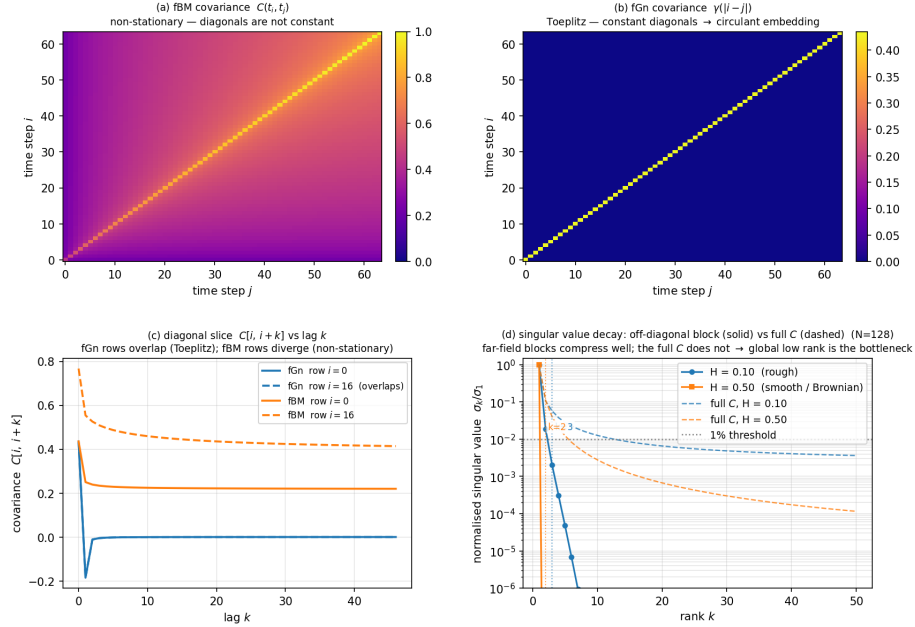


Figure 1: Structural properties of the fBM and fGn covariance matrices ($H = 0.10$). **(a)** fBM covariance $C(t_m, t_n)$: dense, not Toeplitz (variance grows with time). **(b)** fGn covariance $\gamma(|m-n|)$: Toeplitz (constant diagonals). **(c)** Diagonal slice $C[m, m+k]$ vs. lag k : fGn rows for $m = 0$ and $m = N/4$ overlap (Toeplitz); fBM rows diverge (non-stationary). **(d)** Normalised singular values σ_k/σ_1 of the off-diagonal block (solid) and normalised eigenvalues of the full matrix C (dashed) at $H = 0.10$ and $H = 0.50$. The off-diagonal block compresses to rank ≤ 3 at both H ; the full spectrum decays slowly, most slowly at $H = 0.10$. Controlled: $H = 0.10$, $N = 64$ (heatmaps), $N = 128$ (SVD). Independent: process type or Hurst exponent. Dependent: covariance pattern and singular value magnitude.

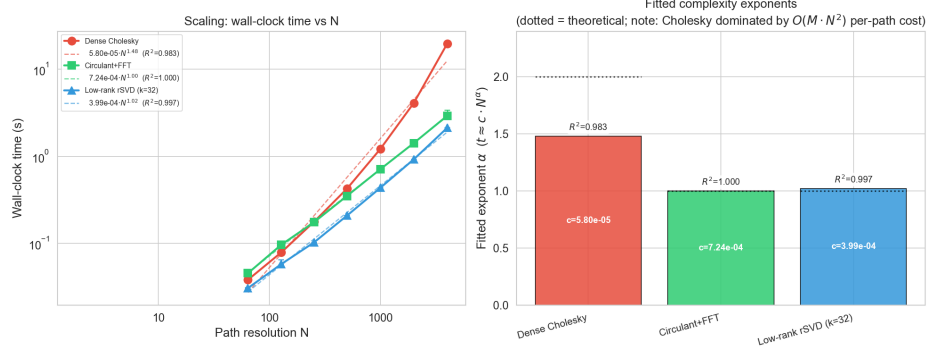


Figure 2: Wall-clock time vs. N on log-log axes (left panel) with fitted power laws $t = c \cdot N^\alpha$, and fitted exponents α with prefactors c (right panel; dotted line = theoretical exponent). Controlled: $M = 10,000$ paths. Independent: $N \in \{64, \dots, 4000\}$, method. Dependent: total wall-clock time (median of 5 repeats; bars: interquartile range).

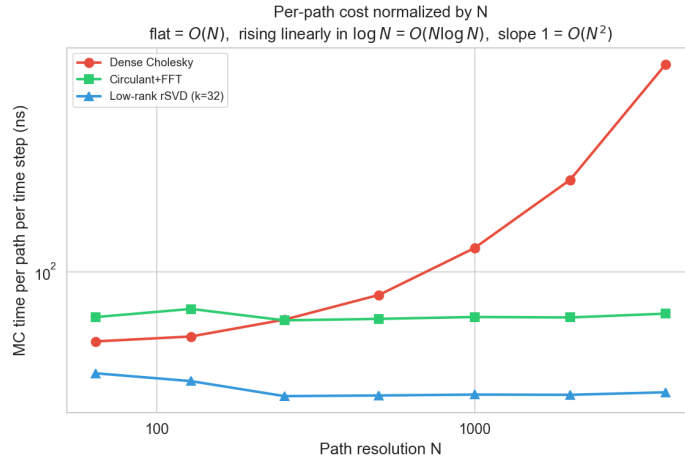


Figure 3: Monte Carlo time per path per time step vs. N . Flat means $O(N)$; rising linearly on the log- x axis means $O(N \log N)$; slope 1 means $O(N^2)$. Controlled: $M = 10,000$ paths. Independent: N , method. Dependent: MC loop time $/(MN)$.

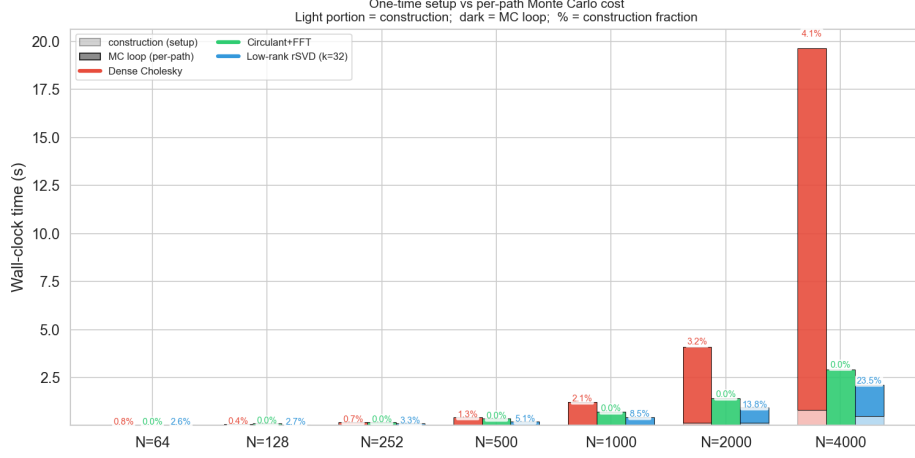


Figure 4: One-time setup vs. per-path Monte Carlo cost breakdown. Light portion = construction (factorization or eigenvalue computation); dark portion = MC loop. Percentages label the construction fraction. The FFT setup is negligible ($< 1\%$); rSVD setup grows with N because forming L_k requires a full $O(N^2k)$ sketch; the Cholesky factorization stays at or below 4% of its runtime because the $O(MN^2)$ MC cost grows just as fast at $M = 10,000$. Controlled: $M = 10,000$ paths. Independent: N , method. Dependent: wall-clock time split between construction and MC loop.

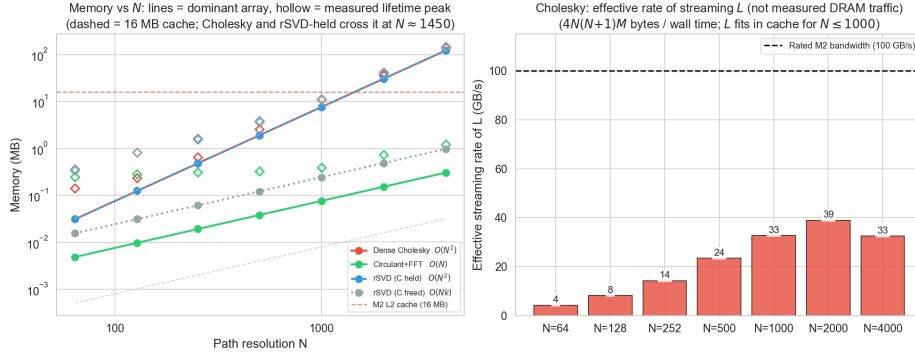


Figure 5: Memory vs. N for all C++ methods (left: lines = dominant array, hollow markers = measured lifetime peak), and the effective rate at which the Cholesky loop streams L (right). The dashed horizontal line marks the M2's 16 MB L2 cache; Cholesky and the rSVD variant that retains C cross it at $N \approx 1450$. Controlled: $M = 10,000$ paths. Independent: N , method. Dependent: memory (MB) and effective streaming rate (GB/s), computed as bytes of L read divided by wall time.

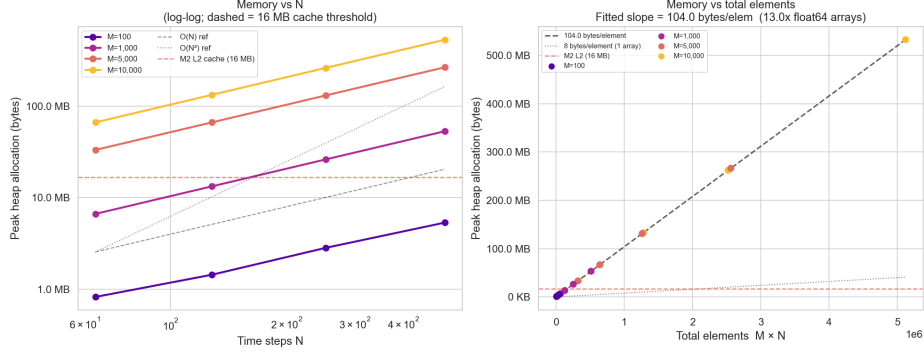


Figure 6: Python RFSV engine peak heap allocation measured by `tracemalloc`. (a) Peak bytes vs. N (log-log) for four path counts M ; the dashed red line marks the M2's 16 MB L2 cache. Scaling is $O(MN)$ (parallel to the $O(N)$ reference), confirming the dominant allocation grows linearly with both M and N . (b) Peak bytes vs. total elements $M \times N$; the fitted slope ≈ 104 bytes/element is 13 float64 words per output element, consistent with the $(M, 2N)$ real and complex working arrays of the path generator. Controlled: $H = 0.10$, $\nu = 0.52$. Independent: $N \in \{64, 128, 252, 512\}$, $M \in \{100, 1000, 5000, 10000\}$. Dependent: peak heap allocation (bytes).

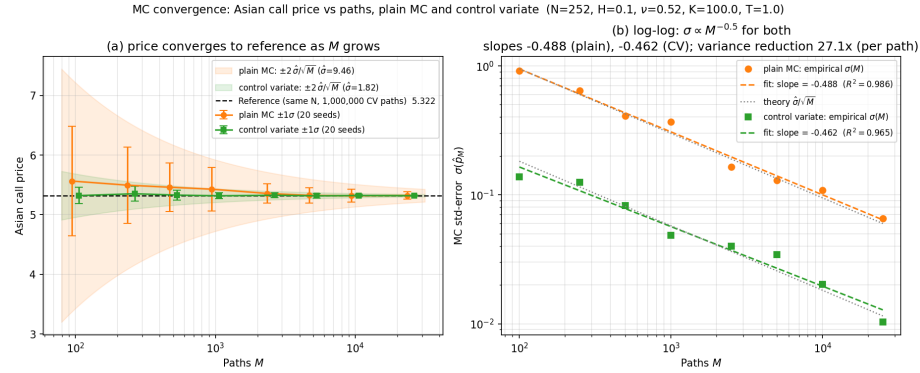


Figure 7: MC convergence of plain Monte Carlo and the control variate (§7.2) on the same paths: price $\pm 1\sigma$ vs. number of paths M (left), and log-log plot of cross-seed standard deviation vs. M with fitted slopes (right). The dashed line is a 10^6 -path control-variate reference on the same $N = 252$ grid; the shaded bands are $\pm 2\hat{\sigma}/\sqrt{M}$ around it, with $\hat{\sigma}$ the per-path standard deviation of each estimator in the reference run. Controlled: $N = 252$, $H = 0.10$, $\nu = 0.52$, $\sigma_0 = 0.2$. Independent: M (left and right); 20 seeds (right). Dependent: mean price $\pm 1\sigma$ and log-log slope.

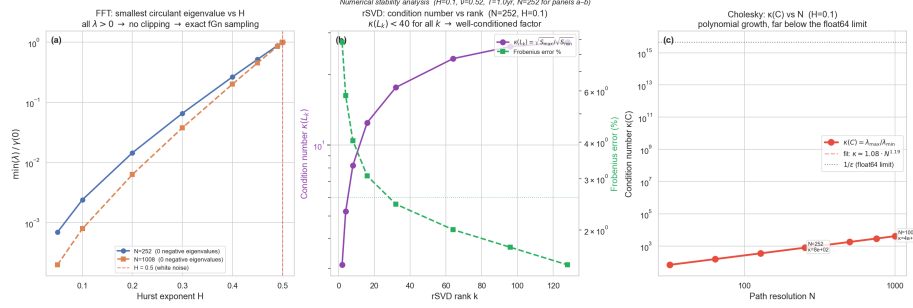


Figure 8: Numerical stability of the three methods. **(a)** FFT: smallest circulant eigenvalue relative to $\gamma(0)$ vs. Hurst exponent H ($N = 252$ and $N = 1008$); no eigenvalue is negative anywhere. **(b)** rSVD: condition number $\kappa(L_k)$ and Frobenius error vs. rank k ($N = 252$, $H = 0.10$). **(c)** Cholesky: condition number $\kappa(C)$ vs. N ($H = 0.10$) with fitted power law.

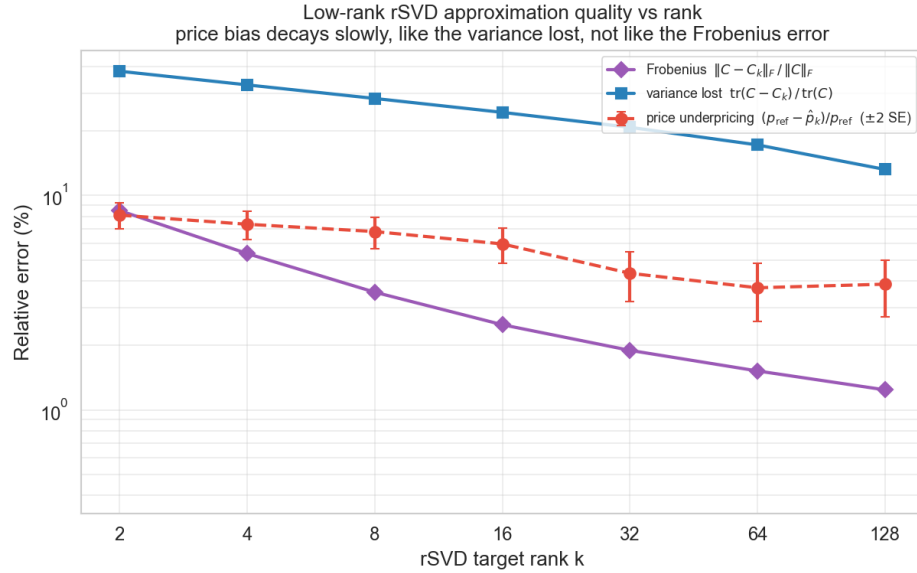


Figure 9: Low-rank rSVD approximation quality vs. rank k : the Frobenius error $\|C - C_k\|_F / \|C\|_F$, the fraction of path variance discarded $\text{tr}(C - C_k) / \text{tr}(C)$ (both exact), and the relative underpricing against the reference with ± 2 standard-error bars. Controlled: $N = 500$, $H = 0.10$, $M = 100,000$ paths per rank. Independent: rank k . Dependent: relative error (%).

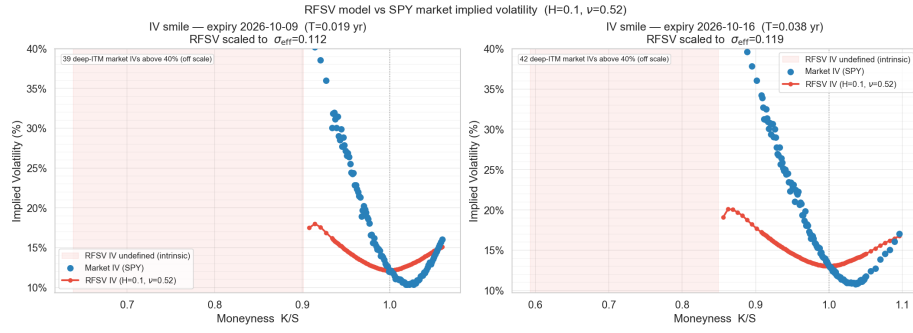


Figure 10: RFSV vs market implied volatility smile for SPY, for the two nearest weekly expirations in a snapshot taken on October 2, 2026 (spot \$771.09). Scatter: market IV computed by Black–Scholes inversion of mid-quote (**brentq**, $r = 0$, filtered to strikes in $[0.60S, 1.40S]$ with valid bid/ask spread); the y -axis is capped at 40%, and the 39 and 42 deep-ITM market IVs above it are marked as off scale. Line: RFSV IV, same inversion applied to Monte Carlo European call prices, with common random numbers across strikes and the sample mean of S_T matched to the forward. The RFSV log-vol level $\mu_0 = \log \sigma_{\text{eff}}$ is calibrated per expiration to match the ATM price (not IV): Brent’s method on the price residual $p_{\text{RFSV}} - p_{\text{mkt}}(\sigma_{\text{ATM}})$. The shaded region marks deep-ITM strikes where no simulated path finishes below K , so the Monte Carlo time value is zero and the IV is undefined. Controlled: $H = 0.10$, $\nu = 0.52$, $r = 0$, $M = 20,000$, $N = 63$ steps per year (at least 5 steps). Independent: strike K , expiration T . Dependent: implied volatility σ_{IV} .

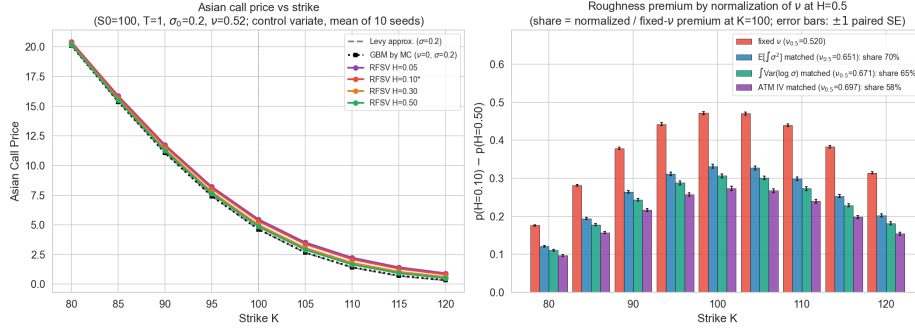


Figure 11: Asian option prices vs strike. **(a)** Price vs K : Lévy (1992) approximation and exact GBM by Monte Carlo at $\sigma = 0.2$ (constant vol), and RFSV at four H values ($\nu = 0.52$, $\sigma_0 = 0.2$); each curve is the mean of 10 seeds of 10,000 paths priced with the control variate, with the same seeds for every curve. **(b)** Roughness premium $\hat{p}(H=0.10) - \hat{p}(H=0.50)$ vs K , at fixed $\nu = 0.52$ and with ν at $H = 0.5$ chosen by three normalizations: matching the expected integrated variance, the integrated log-volatility variance, or the ATM implied volatility; error bars are ± 1 paired standard error across seeds. Controlled: $S_0 = 100$, $T = 1$, $\sigma_0 = 0.2$, $r = 0$, $N = 252$. Independent: K (both panels); H and ν . Dependent: arithmetic Asian call price.

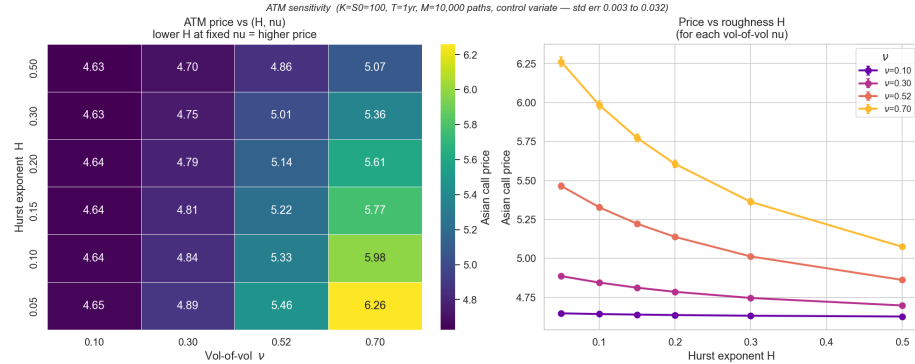


Figure 12: ATM Asian call price vs model parameters. **(a)** Heatmap: price vs (H, ν) at $K = S_0 = 100$ (viridis colormap; low H = rough = lower on the y -axis because the axis is inverted so H increases upward). **(b)** Price vs H for each ν ; curves decrease left-to-right and separate vertically with ν . All cells share the same random numbers and use the control variate; the standard error of each cell ranges from 0.003 ($\nu = 0.10$) to 0.032 ($\nu = 0.70$), and differences between cells are more precise still. Controlled: $S_0 = 100$, $K = 100$, $T = 1$, $\sigma_0 = 0.2$, $r = 0$, $N = 252$, $M = 10,000$. Independent: H , ν . Dependent: ATM Asian call price.

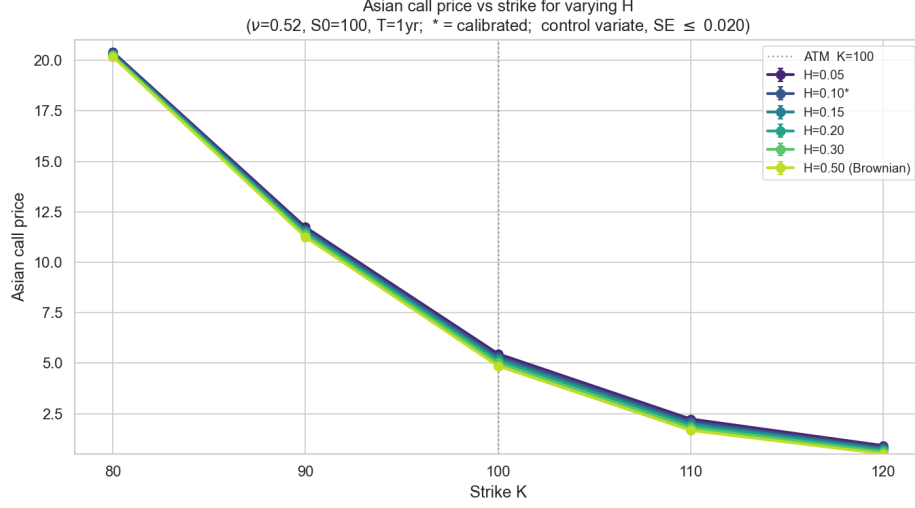


Figure 13: Asian call price vs strike K for varying H at $\nu = 0.52$, with common random numbers across all curves and the control variate. Curves decrease monotonically with strike and are ordered by H at every strike; the spread between $H = 0.05$ and $H = 0.50$ peaks at the money. Controlled: $\nu = 0.52$, $\sigma_0 = 0.2$, $S_0 = 100$, $T = 1$, $r = 0$, $N = 252$, $M = 10,000$. Independent: K , H . Dependent: Asian call price.

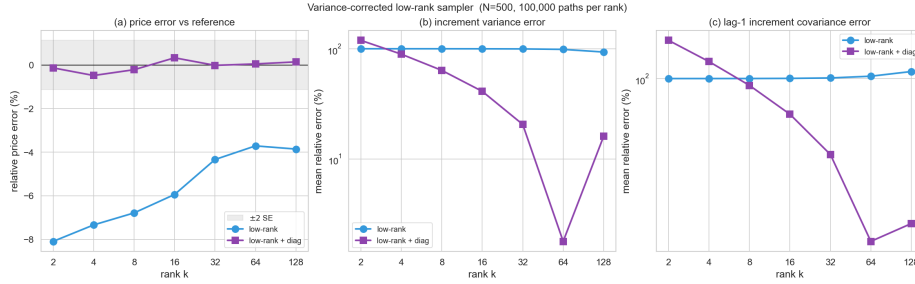


Figure 14: Variance-corrected vs. plain low-rank sampler. **(a)** Price error against the reference (± 2 standard errors shaded). **(b)** Mean relative error of the increment (fGn) variances. **(c)** Mean relative error of the lag-1 increment covariances. Panels (b) and (c) are exact, computed from each sampler's covariance matrix. Controlled: $N = 500$, $H = 0.10$, $\nu = 0.52$, $\sigma_0 = 0.2$, $M = 100,000$ paths per rank. Independent: rank k , sampler. Dependent: relative errors (%).

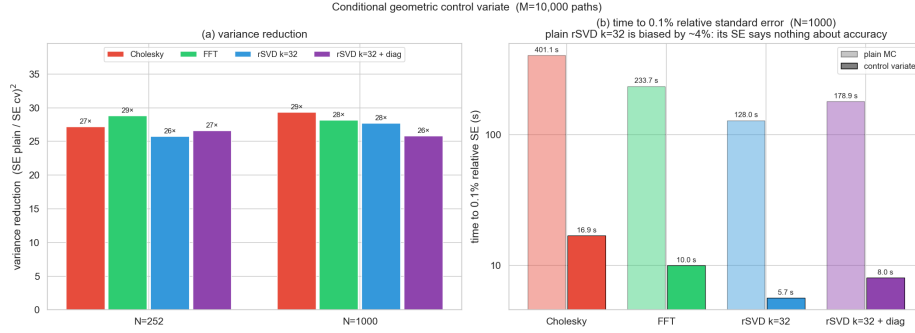


Figure 15: Conditional geometric control variate. **(a)** Variance reduction $(SE_{\text{plain}}/SE_{\text{cv}})^2$ for each sampler. **(b)** Time to a 0.1% relative standard error at $N = 1000$, plain (light) and control variate (dark), on a log scale. Controlled: $H = 0.10$, $\nu = 0.52$, $\sigma_0 = 0.2$, $M = 10,000$. Independent: sampler, N , estimator. Dependent: variance reduction and time (s).

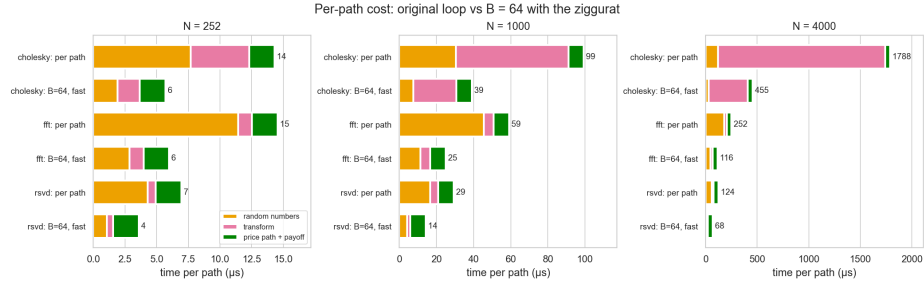


Figure 16: Time per path split into its three stages: drawing the normals, the transform (mat-vec, FFT or $L_k z$; normals replayed from a buffer), and the price path with its payoff. For each sampler, the upper bar is the original per-path loop with `std::mt19937`, the lower bar the batched version ($B = 64$) with the ziggurat generator. Each stage is timed in isolation, so the bars understate the end-to-end times of Figure 17. Controlled: $k = 32$, one thread. Independent: N , sampler, configuration. Dependent: time per path (μs).

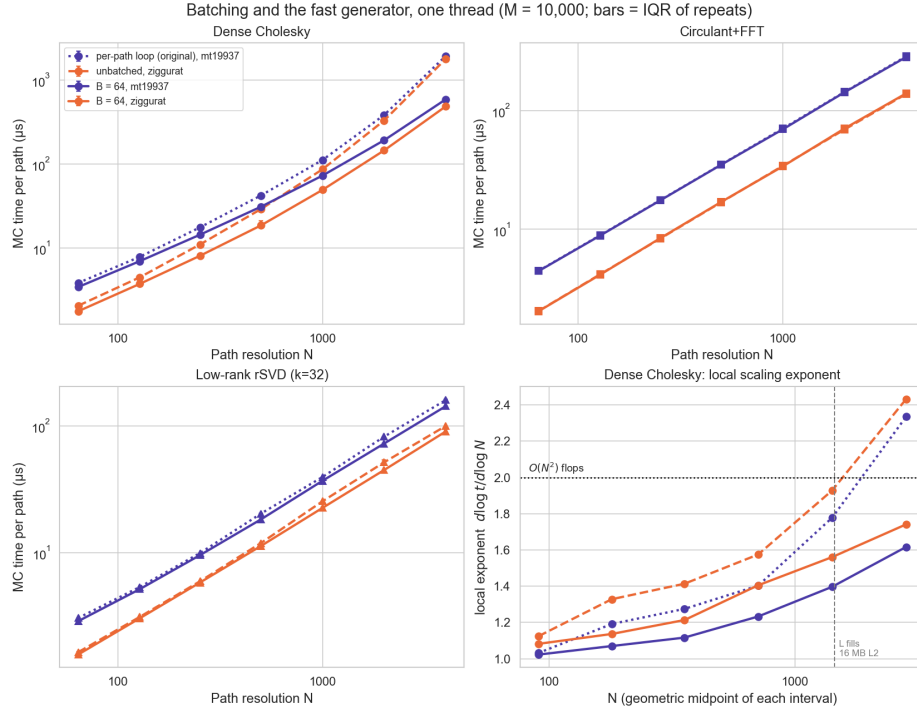


Figure 17: MC time per path vs N for four configurations of each sampler, one thread: the original loop (dotted), unbatched with the zigurat (dashed), and $B = 64$ with either generator (solid). Color encodes the generator. Bottom right: local scaling exponent of the Cholesky time between neighbouring N . Controlled: $M = 10,000$, $k = 32$, one thread. Independent: N , block size, generator. Dependent: MC time per path (median of 3 repeats; bars: interquartile range).

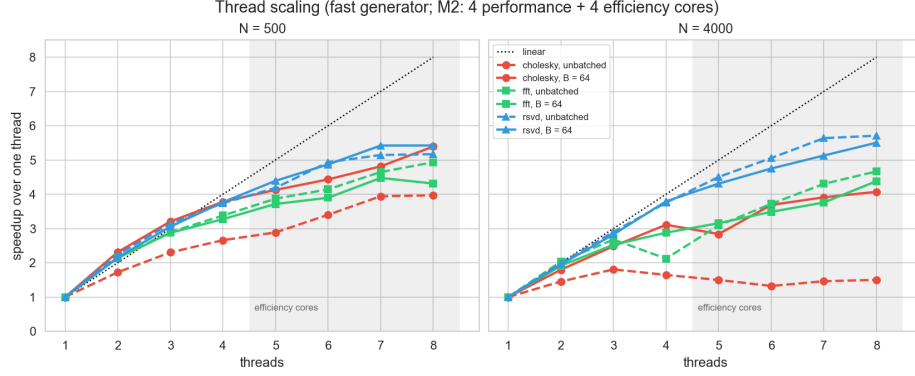


Figure 18: Speedup over one thread vs thread count, unbatched (dashed; the FFT pairs two paths per transform) and $B = 64$ (solid), with the fast generator. The shaded region uses the M2's efficiency cores. Controlled: $M = 10,000$ ($N = 500$) or 2,000 ($N = 4000$), fast generator. Independent: threads, N , block size. Dependent: speedup of the MC time (median of 3 repeats).

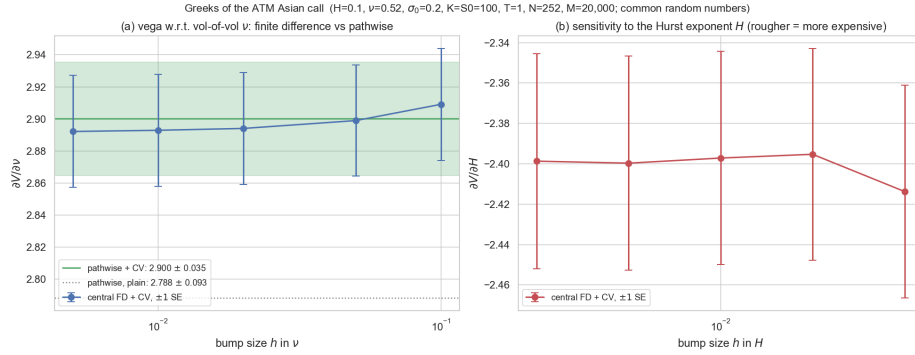


Figure 19: Greeks of the ATM Asian call. **(a)** $\partial V / \partial \nu$ by central difference with common random numbers and the control variate, against bump size h , and the pathwise estimator (18) with and without its differentiated control. **(b)** $\partial V / \partial H$ by central difference. Error bars are ± 1 standard error from the per-path differences. Controlled: $H = 0.10$, $\nu = 0.52$, $\sigma_0 = 0.2$, $K = S_0 = 100$, $T = 1$, $r = 0$, $N = 252$, $M = 20,000$. Independent: bump size h , estimator. Dependent: $\partial V / \partial \nu$, $\partial V / \partial H$.